THE PREMIUM INTERVIEW NOTEBOOK SERIES ✦ JAVA EDITION

120 SOLVED PROBLEMS · 24 TOPICS · JAVA 17

ZERO TO HERO

JAVA Programming Interviews

Handwritten-style coding notes with clear solutions & diagrams

✓

O(n)

```
for (int idea = 0; idea < 120; idea++) {
  solve(); explain(); verify();
}
```

{ }

- ✓ Unchanged interview questions
- ✓ 120 answered follow-ups
- ✓ Code & walkthroughs
- ✓ 30-day practice roadmap

PRACTISE THE PATTERN. EXPLAIN THE REASON. PASS THE CODE REVIEW.

START HERE

How to study this notebook

Each problem keeps its **original question word for word**. Answers are rewritten in plain English, but algorithms, Java methods, contracts and limitations stay grounded in the supplied guide.

1. TRY

Read only the question and example. Explain your first idea.

2. TRACE

Walk through the sketch and a boundary input.

3. CODE

Run the Java 17 method and read its key lines.

4. DEFEND

Explain complexity, pitfalls, and the follow-up answer.

What's in the edition?

120	120	24	124
unchanged coding questions	answered follow-ups	topic families	retested example assertions

Important Java contract

Code is designed for **Java 17**, not Java 8-only syntax. Some problems use records, modern collections and Unicode code-point APIs. Unless a problem says otherwise, inputs and elements are assumed non-null. Complexity and edge cases follow the original problem contract.

> The 120 source methods compiled successfully and 124 extracted example assertions passed in this edition. The original guide reports a larger 4,479-assertion test suite; that full original test runner was not available in the attached PDF and was not independently rerun.

Adapted for study from the supplied 101-page reference. Check licensing and resale rights for all underlying source content before commercial distribution.

TOPIC MAP

24 chapters · 120 solved problems

Five different questions per topic. Find the matching problem number in the page heading.

01	Number foundations	Q001–Q005
02	Digits, bits and arithmetic	Q006–Q010
03	String foundations	Q011–Q015
04	String search and sliding windows	Q016–Q020
05	Array transformations	Q021–Q025
06	Hashing and frequency problems	Q026–Q030
07	Two pointers and prefix/suffix state	Q031–Q035
08	Binary search patterns	Q036–Q040
09	Sorting, selection and intervals	Q041–Q045
10	Matrices and grid traversal	Q046–Q050
11	Stacks and monotonic stacks	Q051–Q055
12	Heaps, windows and cache design	Q056–Q060
13	Linked-list foundations	Q061–Q065
14	Advanced linked-list problems	Q066–Q070
15	Binary-tree traversal and structure	Q071–Q075
16	BSTs and tree serialization	Q076–Q080
17	Graphs and reachability	Q081–Q085
18	Recursion and backtracking	Q086–Q090
19	Dynamic programming foundations	Q091–Q095
20	Dynamic programming on sequences and grids	Q096–Q100
21	Java streams and grouping	Q101–Q105
22	Collectors, ordering and reductions	Q106–Q110
23	Practical Java correctness	Q111–Q115
24	Concurrent programming tasks	Q116–Q120

"Explain the invariant before you write the loop."

Q001. Check whether an integer is prime.

✎ SIMPLE ANSWER

Reject values below two; check divisors only through the square root. Use division in the loop condition to avoid overflow in d*d.

SKETCH THE PATTERN

EXAMPLE / INPUT
Check whether an integer is prime.

→

KEY LOGIC
Reject values below two; check divisors only through the square root.

→

RESULT
true; 1 -> false

💡 If n has a nontrivial divisor, one factor is at most sqrt(n).

JAVA 17 · SOLUTION — Source Q001 · 5 lines

```
static boolean isPrime(int n) {
  if (n < 2) return false;
  for (int d = 2; d <= n / d; d++) if (n % d == 0) return false;
  return true;
}
```

⏱ TIME & SPACE

Time O(sqrt(n)); auxiliary space O(1).

⚠ INTERVIEW PITFALL

Negative values, zero and one are not prime.

FOLLOW-UP QUESTION · ANSWERED

How would a sieve answer many prime queries?

Use the Sieve of Eratosthenes once up to the largest query number. Mark multiples of each prime starting at p*p; then each query is a constant-time array lookup. Preprocessing takes about O(N log log N) time and O(N) space.

Q002. Find the greatest common divisor.

✎ SIMPLE ANSWER

For nonnegative longs, repeatedly replace the pair by the divisor and remainder. Euclid preserves the gcd and stops when the remainder becomes zero.

SKETCH THE PATTERN

EXAMPLE / INPUT
gcd(48,18)

→

KEY LOGIC
For nonnegative longs, repeatedly replace the pair by the divisor and remainder.

→

RESULT
6; gcd(0,0) -> 0

💡 (48,18) -> (18,12) -> (12,6) -> (6,0).

JAVA 17 · SOLUTION — Source Q002 · 5 lines

```
static long gcd(long a, long b) {
  if (a < 0 || b < 0) throw new IllegalArgumentException();
  while (b != 0) { long r = a % b; a = b; b = r; }
  return a;
}
```

⏱ TIME & SPACE

Time O(log(min(a,b))) for positive operands; space O(1).

⚠ INTERVIEW PITFALL

The nonnegative contract avoids abs(Long.MIN_VALUE). Zero is handled explicitly by the loop.

FOLLOW-UP QUESTION · ANSWERED

How would you compute lcm without overflowing?

Compute gcd first, then calculate (a / gcd(a,b)) * b. Dividing before multiplying reduces overflow risk. For guaranteed exact large results, use BigInteger; Math.multiplyExact can detect long overflow.

Q003. Compute factorial without primitive overflow.

✎ SIMPLE ANSWER

Multiply into BigInteger rather than int or long. Reject negative input and define zero factorial as one.

SKETCH THE PATTERN

EXAMPLE / INPUT
Compute factorial without primitive overflow.

→

KEY LOGIC
Multiply into BigInteger rather than int or long.

→

RESULT
120; 0 -> 1

💡 The accumulator contains i! after processing i.

JAVA 17 · SOLUTION — Source Q003 · 6 lines

```
static BigInteger factorial(int n) {
  if (n < 0) throw new IllegalArgumentException();
  BigInteger result = BigInteger.ONE;
  for (int i = 2; i <= n; i++) result = result.multiply(BigInteger.valueOf(i));
  return result;
}
```

⏱ TIME & SPACE

n-1 big-integer multiplications; space proportional to result size. Bit cost grows with operand size.

⚠ INTERVIEW PITFALL

Do not claim BigInteger arithmetic is constant-time.

FOLLOW-UP QUESTION · ANSWERED

When could a long implementation use Math.multiplyExact?

Use a long result while each factorial step fits in signed 64-bit range. Math.multiplyExact(result, i) throws ArithmeticException when it would overflow. For larger n, switch to BigInteger; 20! fits in a long but 21! does not.

Q004. Compute the nth Fibonacci number.

✎ SIMPLE ANSWER

Maintain two adjacent values and advance them iteratively. BigInteger gives exact results beyond primitive limits without exponential recursive recomputation.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
F(10)	Maintain two adjacent values and advance them iteratively.	55; F(0) -> 0

💡 At loop entry i, a=F(i) and b=F(i+1).

JAVA 17 · SOLUTION — Source Q004 · 8 lines

```
static BigInteger fibonacci(int n) {
  if (n < 0) throw new IllegalArgumentException();
  BigInteger a = BigInteger.ZERO, b = BigInteger.ONE;
  for (int i = 0; i < n; i++) {
    BigInteger next = a.add(b); a = b; b = next;
  }
  return a;
}
```

⏱ TIME & SPACE

n additions; O(n) bits of live numeric storage, with growing addition bit cost.

⚠ INTERVIEW PITFALL

Reject negative index; define the zero-index convention before coding.

FOLLOW-UP QUESTION · ANSWERED

How would fast doubling reduce the number of arithmetic steps?

Fast doubling calculates F(2k)=F(k)*(2*F(k+1)-F(k)) and F(2k+1)=F(k)^2+F(k+1)^2. Halve the index recursively or use its bits, taking O(log n) big-integer arithmetic steps instead of O(n).

Q005. Check whether a nonnegative integer is a decimal palindrome.

✎ SIMPLE ANSWER

Reverse the digits into a long and compare with the original. The wider accumulator safely holds the reversed decimal representation of any nonnegative int.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
1221	Reverse the digits into a long and compare with the original.	true; -121 -> false

💡 Reversing 1221 reconstructs the same value; trailing zeros change the representation.

JAVA 17 · SOLUTION — Source Q005 · 8 lines

```
static boolean numberPalindrome(int n) {
  if (n < 0) return false;
  long reversed = 0; int remaining = n;
  while (remaining != 0) {
    reversed = reversed * 10 + remaining % 10; remaining /= 10;
  }
  return reversed == n;
}
```

⏱ TIME & SPACE

Time O(decimal digits); space O(1).

⚠ INTERVIEW PITFALL

Zero is a palindrome; leading zeros are not part of an integer value.

FOLLOW-UP QUESTION · ANSWERED

How would reversing only half the digits work?

Build only the reversed last half. Stop when the remaining half is no larger than the reversed half. Compare them directly for even digit counts, or drop the reversed half's middle digit for odd counts. Numbers ending in zero need special handling.

Q006. Sum the decimal digits of a signed integer.

✎ SIMPLE ANSWER

Convert to long before taking the absolute value. This handles Integer.MIN_VALUE, whose magnitude does not fit in an int.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
-123	Convert to long before taking the absolute value.	6; MIN_VALUE -> 47

💡 The sign does not contribute a digit.

JAVA 17 · SOLUTION — Source Q006 · 5 lines

```
static int digitSum(int value) {
  long n = Math.abs((long) value); int sum = 0;
  while (n > 0) { sum += n % 10; n /= 10; }
  return sum;
}
```

⏱ TIME & SPACE

Time O(decimal digits); space O(1).

⚠ INTERVIEW PITFALL

Math.abs(int MIN_VALUE) remains negative; widening must happen first.

FOLLOW-UP QUESTION · ANSWERED

How would the same idea generalize to base b?

Replace % 10 and / 10 with % b and / b, with b >= 2. Repeated remainders are the base-b digits; sum them until the magnitude becomes zero. Use long before taking the absolute value of a signed int.

Q007. Check an Armstrong number.

✎ SIMPLE ANSWER

Count decimal digits, raise each digit to that count using integer multiplication, and compare the sum with the number. For an int input the accumulated powers fit in a long.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
153	Count decimal digits, raise each digit to that count using integer multiplication, and compare the sum with the number.	true because 1^3+5^3+3^3=153

💡 Use integer powers rather than floating-point equality.

JAVA 17 · SOLUTION — Source Q007 · 10 lines

```
static boolean armstrong(int value) {
  if (value < 0) return false;
  int digits = Integer.toString(value).length(), n = value; long sum = 0;
  do {
    int d = n % 10; long power = 1;
    for (int i = 0; i < digits; i++) power *= d;
    sum += power; n /= 10;
  } while (n > 0);
  return sum == value;
}
```

⏱ TIME & SPACE

Time O(digits squared); auxiliary space O(digits) for the digit-count string.

⚠ INTERVIEW PITFALL

Zero has one decimal digit and passes; negative inputs fail.

FOLLOW-UP QUESTION · ANSWERED

How would you avoid the digit-count string allocation?

Count digits with a division loop: start at one and repeatedly divide by ten until the remaining number is below ten. Then perform the power sum. This avoids Integer.toString and its allocation.

Q008. Reverse an int and detect overflow.

✎ SIMPLE ANSWER

Use a long accumulator, apply the original sign, and return OptionalInt.empty if the reversed value lies outside int range. This separates invalid output from the valid value zero.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
-120	Use a long accumulator, apply the original sign, and return OptionalInt.empty if the reversed value lies outside int range.	-21; 1534236469 -> empty

💡 The decimal reversal is checked before narrowing.

JAVA 17 · SOLUTION — Source Q008 · 8 lines

```
static OptionalInt reverseInt(int value) {
  long n = Math.abs((long) value), reversed = 0;
  while (n > 0) { reversed = reversed * 10 + n % 10; n /= 10; }
  if (value < 0) reversed = -reversed;
  if (reversed < Integer.MIN_VALUE || reversed > Integer.MAX_VALUE)
    return OptionalInt.empty();
  return OptionalInt.of((int) reversed);
}
```

⏱ TIME & SPACE

Time O(decimal digits); space O(1).

⚠ INTERVIEW PITFALL

Zero is present with value zero; an overflowing result is absent.

FOLLOW-UP QUESTION · ANSWERED

How would you check bounds without a long accumulator?

Before updating rev = rev*10 + digit, check whether rev exceeds MAX/10 or equals MAX/10 with too-large digit; mirror the check against MIN for negatives. Reject before multiplication rather than relying on overflow after it happens.

Q009. Implement exponentiation by squaring.

✎ SIMPLE ANSWER

Square the base while halving the exponent; multiply the answer when the current exponent bit is set. Convert exponent to long so negating Integer.MIN_VALUE is safe.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
2^10	Square the base while halving the exponent; multiply the answer when the current exponent bit is set.	1024; 2^-3 -> 0.125

💡 Each bit selects one power-of-two factor.

JAVA 17 · SOLUTION — Source Q009 · 10 lines

```
static double power(double base, int exponent) {
  long n = exponent;
  if (n < 0) { base = 1.0 / base; n = -n; }
  double result = 1.0;
  while (n > 0) {
    if ((n & 1) != 0) result *= base;
    base *= base; n >>= 1;
  }
  return result;
}
```

⏱ TIME & SPACE

Time O(log(|exponent|+1)); space O(1).

⚠ INTERVIEW PITFALL

IEEE floating-point rounding/infinity apply; this convention returns one for exponent zero.

FOLLOW-UP QUESTION · ANSWERED

What changes for modular integer exponentiation?

Apply the same squaring loop but reduce every multiplication modulo positive m: result=(result*base)%m and base=(base*base)%m. Use long or BigInteger multiplication to avoid overflow; negative exponents require modular inverses when they exist.

Q010. Find the single value when every other int occurs twice.

✎ SIMPLE ANSWER

XOR all values: equal pairs cancel and zero is the identity. This relies on the stated multiplicity contract; it does not validate arbitrary input counts.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[4,1,2,1,2]	XOR all values: equal pairs cancel and zero is the identity.	Associativity and commutativity allow cancellation regardless of ord

💡 Associativity and commutativity allow cancellation regardless of order.

JAVA 17 · SOLUTION — Source Q010 · 5 lines

```
static int singleNumber(int[] values) {
  int result = 0;
  for (int value : values) result ^= value;
  return result;
}
```

⏱ TIME & SPACE

Time O(n); space O(1).

⚠ INTERVIEW PITFALL

Input must contain exactly one unpaired value; empty input violates that problem contract.

FOLLOW-UP QUESTION · ANSWERED

How would the state change if every other value occurred three times?

Track bit counts modulo three across 32 bit positions, or use two bit masks ('ones' and 'twos') that form a modulo-3 counter. The leftover bits reconstruct the value occurring once, including negative int values.

Q011. Reverse a string without splitting Unicode code points.

✎ SIMPLE ANSWER

Convert to code points and append them in reverse. This preserves supplementary characters, but reverses code points rather than user-perceived grapheme clusters.

SKETCH THE PATTERN

EXAMPLE / INPUT
"A[U+1F600]B"

→

KEY LOGIC
Convert to code points and append them in reverse.

→

RESULT
"B[U+1F600]A"

💡 The emoji is one code point represented by two UTF-16 code units.

JAVA 17 · SOLUTION — Source Q011 · 6 lines

```
static String reverseCodePoints(String text) {
  int[] points = text.codePoints().toArray();
  StringBuilder result = new StringBuilder();
  for (int i = points.length - 1; i >= 0; i--) result.appendCodePoint(points[i]);
  return result.toString();
}
```

⏱ TIME & SPACE

Time O(n UTF-16 units); space O(n), including output.

⚠ INTERVIEW PITFALL

Combining marks and emoji sequences may span several code points.

FOLLOW-UP QUESTION · ANSWERED

What library-level segmentation would grapheme reversal need?

Use a Unicode grapheme-cluster segmenter (ICU4J BreakIterator for extended grapheme clusters) and reverse the resulting clusters. Reversing code points alone may separate accents, skin-tone modifiers, or joined emoji.

Q012. Check an exact code-point palindrome.

✎ SIMPLE ANSWER

Compare mirrored Unicode code points. This contract is case-sensitive and does not discard spaces, punctuation or combining marks; normalization is a separate requirement.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
"a[U+1F600]a"	Compare mirrored Unicode code points.	true; "Aa" -> false

💡 Every mirrored pair must be equal.

JAVA 17 · SOLUTION — Source Q012 · 6 lines

```
static boolean textPalindrome(String text) {
  int[] points = text.codePoints().toArray();
  for (int l = 0, r = points.length - 1; l < r; l++, r--)
    if (points[l] != points[r]) return false;
  return true;
}
```

⏱ TIME & SPACE

Time O(n); auxiliary space O(n) for code points.

⚠ INTERVIEW PITFALL

An empty string passes; normalized/case-folded palindrome is another contract.

FOLLOW-UP QUESTION · ANSWERED

How would you define an alphanumeric-only Unicode version?

Iterate Unicode code points, keep only Character.isLetterOrDigit values, normalize case with a defined Unicode strategy, and compare the remaining sequence from both ends. Specify whether accents and normalization are significant.

Q013. Check whether two lowercase-English strings are anagrams.

✎ SIMPLE ANSWER

Count the 26 lowercase letters in one string and subtract counts for the other. Validate the alphabet explicitly rather than indexing arbitrary characters into a fixed-size table.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
"listen", "silent"	Count the 26 lowercase letters in one string and subtract counts for the other.	true

💡 Equal multisets produce a zero difference vector.

JAVA 17 · SOLUTION — Source Q013 · 13 lines

```
static boolean anagramLowercase(String a, String b) {
  int[] counts = new int[26];
  for (char c : a.toCharArray()) {
    if (c < 'a' || c > 'z') throw new IllegalArgumentException();
    counts[c - 'a']++;
  }
  for (char c : b.toCharArray()) {
    if (c < 'a' || c > 'z') throw new IllegalArgumentException();
    counts[c - 'a']--;
  }
  for (int count : counts) if (count != 0) return false;
  return true;
}
```

⏱ TIME & SPACE

Time O(n+m); auxiliary space O(n+m) from toCharArray copies; counts use O(1).

⚠ INTERVIEW PITFALL

Case, punctuation and full Unicode are outside this alphabet contract.

FOLLOW-UP QUESTION · ANSWERED

How would a code-point frequency map broaden the contract?

Iterate codePoints() and update Map<Integer,Integer> counts rather than using a 26-element array. This supports non-English letters and emoji. Decide first whether case, canonical normalization and whitespace should count.

Q014. Find the first non-repeated Unicode code point.

✎ SIMPLE ANSWER

Count with an insertion-ordered map, then return the first entry whose count is one. Return OptionalInt so an absent result is distinct from any valid code point.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
"swiss"	Count with an insertion-ordered map, then return the first entry whose count is one.	code point for w

💡 Insertion order preserves first occurrence order during counting.

JAVA 17 · SOLUTION — Source Q014 · 7 lines

```
static OptionalInt firstUnique(String text) {
  Map<Integer,Integer> counts = new LinkedHashMap<>();
  text.codePoints().forEach(c -> counts.merge(c, 1, Integer::sum));
  for (var entry : counts.entrySet())
    if (entry.getValue() == 1) return OptionalInt.of(entry.getKey());
  return OptionalInt.empty();
}
```

⏱ TIME & SPACE

Expected time O(n); space O(u) distinct code points.

⚠ INTERVIEW PITFALL

Map hashing cost is expected; no result for empty or all-repeated input.

FOLLOW-UP QUESTION · ANSWERED

Could two passes over the original string avoid insertion-order storage?

Yes. The first pass builds counts in a regular HashMap; the second scans original code points and returns the first with count one. You save insertion-order bookkeeping but scan the input twice.

Q015. Run-length encode lowercase letters.

✎ SIMPLE ANSWER

Scan each maximal run and append its letter plus count. Restrict input to lowercase letters so the encoding is unambiguous with decimal counts.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
"aaabbc"	Scan each maximal run and append its letter plus count.	"a3b2c1"

💡 Run aa a has length three; the next run starts at the first different letter.

JAVA 17 · SOLUTION — Source Q015 · 11 lines

```
static String runLength(String text) {
  StringBuilder result = new StringBuilder();
  for (int i = 0; i < text.length();) {
    char c = text.charAt(i);
    if (c < 'a' || c > 'z') throw new IllegalArgumentException();
    int j = i + 1;
    while (j < text.length() && text.charAt(j) == c) j++;
    result.append(c).append(j - i); i = j;
  }
  return result.toString();
}
```

⏱ TIME & SPACE

Time O(n); space O(output size).

⚠ INTERVIEW PITFALL

A single letter becomes letter plus one; empty input returns empty.

FOLLOW-UP QUESTION · ANSWERED

How would you encode input containing digits or escape characters?

Use a self-delimiting format: encode each run as its numeric length followed by a separator and the symbol, or escape reserved characters. Define how decoding parses multi-digit lengths so text containing digits is unambiguous.

Q016. Find the longest substring without repeated code points.

✎ SIMPLE ANSWER

Use a sliding window and store each code point's latest position. Move the left boundary forward past the repeated occurrence, never backward.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
"abcabcbb"	Use a sliding window and store each code point's latest position.	3; "[U+1F600]a[U+1F600]" -> 2

💡 For abba, the last a must not move left backward after handling repeated b.

JAVA 17 · SOLUTION Source Q016 · 10 lines

```
static int longestUnique(String text) {
  int[] a = text.codePoints().toArray(); Map<Integer,Integer> last = new HashMap<>();
  int left = 0, best = 0;
  for (int right = 0; right < a.length; right++) {
    Integer old = last.put(a[right], right);
    if (old != null) left = Math.max(left, old + 1);
    best = Math.max(best, right - left + 1);
  }
  return best;
}
```

⏱ TIME & SPACE

Expected time O(n); space O(n) including the code-point array.

⚠ INTERVIEW PITFALL

Length is not a UTF-16 index or a grapheme count.

FOLLOW-UP QUESTION · ANSWERED

How would you return the corresponding substring safely?

Store the best window's starting and ending code-point offsets. Convert those offsets to UTF-16 boundaries with text.offsetByCodePoints, then use substring. Never treat code-point indices as char indices.

Q017. Find the maximum vowels in a fixed-length substring.

✎ SIMPLE ANSWER

Count English vowels in the first window, then remove the departing character and add the arriving one. This example recognizes lower- and uppercase English vowels and requires a valid window length.

SKETCH THE PATTERN

EXAMPLE / INPUT
"abciiidef", k=3

→

KEY LOGIC
Count English vowels in the first window, then remove the departing character and add the arriving one.

→

RESULT
A fixed-size window needs only two count changes per movement.

💡 A fixed-size window needs only two count changes per movement.

JAVA 17 · SOLUTION Source Q017 · 12 lines

```
static int maxVowels(String text, int k) {
  if (k < 0 || k > text.length()) throw new IllegalArgumentException();
  int count = 0, best;
  for (int i = 0; i < k; i++) if ("aeiouAEIOU".indexOf(text.charAt(i)) >= 0) count++;
  best = count;
  for (int i = k; i < text.length(); i++) {
    if ("aeiouAEIOU".indexOf(text.charAt(i)) >= 0) count++;
    if ("aeiouAEIOU".indexOf(text.charAt(i-k)) >= 0) count--;
    best = Math.max(best, count);
  }
  return best;
}
```

⏱ TIME & SPACE

Time O(n); space O(1).

⚠ INTERVIEW PITFALL

Window indices use UTF-16 units; k=0 returns zero.

FOLLOW-UP QUESTION · ANSWERED

How would you generalize to any predicate or code points?

Replace the hard-coded vowel check with an IntPredicate and count matching code points. Convert text to a code-point array, slide a k-code-point window, and update the count for the exiting and entering code point.

Q018. Find a longest palindromic substring.

✎ SIMPLE ANSWER

Expand around each odd and even center. Keep the first encountered longest result by updating only for strictly greater length.

SKETCH THE PATTERN

EXAMPLE / INPUT
"babad"

→

KEY LOGIC
Expand around each odd and even center.

→

RESULT
"bab" with this tie rule

💡 Every palindrome has either one center character or one center gap.

JAVA 17 · SOLUTION — Source Q018 · 13 lines

```
static String longestPalindrome(String text) {
  int start = 0, length = 0;
  for (int center = 0; center < text.length(); center++) {
    for (int even = 0; even <= 1; even++) {
      int l = center, r = center + even;
      while (l >= 0 && r < text.length() && text.charAt(l) == text.charAt(r)) {
        if (r-l+1 > length) { start = l; length = r-l+1; }
        l--; r++;
      }
    }
  }
  return text.substring(start, start + length);
}
```

⏱ TIME & SPACE

Time O(n squared); auxiliary space O(1), plus returned substring.

⚠ INTERVIEW PITFALL

Empty input returns empty; code-point semantics require a different representation.

FOLLOW-UP QUESTION · ANSWERED

How does Manacher's algorithm reduce repeated expansion?

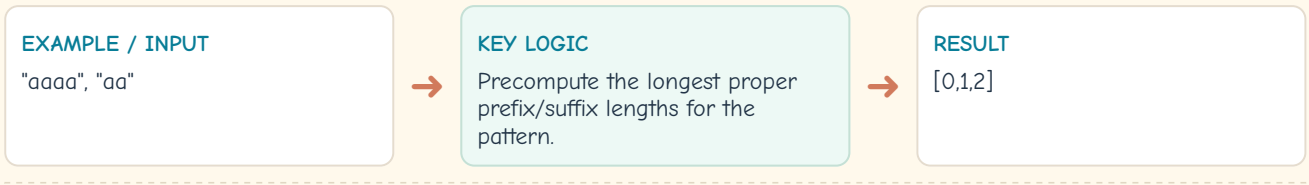
Manacher's algorithm reuses palindrome radii around mirrored centers and expands only when needed. Insert conceptual separators to handle odd/even lengths uniformly. It finds the longest palindrome in O(n) time.

Q019. Find all occurrences with KMP, including overlaps.

✎ SIMPLE ANSWER

Precompute the longest proper prefix/suffix lengths for the pattern. On mismatch, reuse that information instead of restarting the text scan.

SKETCH THE PATTERN



💡 After a match, falling back to the proper suffix preserves overlapping candidates.

JAVA 17 · SOLUTION Source Q019 · 16 lines

```
static List<Integer> occurrences(String text, String pattern) {
  List<Integer> result = new ArrayList<>(); int m = pattern.length();
  if (m == 0) { for (int i=0; i<=text.length(); i++) result.add(i); return result; }
  int[] prefix = new int[m];
  for (int i=1, j=0; i<m; i++) {
    while (j>0 && pattern.charAt(i)!=pattern.charAt(j)) j=prefix[j-1];
    if (pattern.charAt(i)==pattern.charAt(j)) j++;
    prefix[i]=j;
  }
  for (int i=0, j=0; i<text.length(); i++) {
    while (j>0 && text.charAt(i)!=pattern.charAt(j)) j=prefix[j-1];
    if (text.charAt(i)==pattern.charAt(j)) j++;
    if (j==m) { result.add(i-m+1); j=prefix[j-1]; }
  }
  return result;
}
```

⏱ TIME & SPACE

Time O(n+m); auxiliary space O(m), excluding match output.

⚠ INTERVIEW PITFALL

Empty-pattern behavior is specified here, not left accidental.

FOLLOW-UP QUESTION · ANSWERED

What changes for a streaming text source?

Keep the KMP prefix table and current matched-prefix length between chunks. Process each new character without resetting the matcher; maintain an absolute stream position to report matches, including those crossing a chunk boundary.

Q020. Find the minimum window containing a target multiset.

✎ SIMPLE ANSWER

For ASCII input, track remaining required occurrences. Expand until all are covered, then shrink while coverage remains valid.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
"ADOBECODEBANC", "ABC"	For ASCII input, track remaining required occurrences.	"BANC"

💡 Negative counts represent surplus characters; shrinking stops when a required occurrence is lost.

JAVA 17 · SOLUTION Source Q020 · 15 lines

```
static String minWindow(String text, String target) {
  for (int i=0;i<text.length();i++) if (text.charAt(i)>=128) throw new IllegalArgumentException();
  int[] need = new int[128];
  for (char c:target.toCharArray()) { if(c>=128) throw new IllegalArgumentException(); need[c]+
+; }
  if (target.isEmpty()) return "";
  int missing=target.length(), left=0, start=0, length=Integer.MAX_VALUE;
  for(int right=0;right<text.length();right++) {
    if(need[text.charAt(right)]-- > 0) missing--;
    while(missing==0) {
      if(right-left+1<length) { start=left; length=right-left+1; }
      if(++need[text.charAt(left++)] > 0) missing++;
    }
  }
  return length==Integer.MAX_VALUE ? "" : text.substring(start,start+length);
}
```

⏱ TIME & SPACE

Time O(n+m); auxiliary space O(m) from target character copy, fixed count table.

⚠ INTERVIEW PITFALL

Empty target/no cover returns empty; this method explicitly rejects non-ASCII.

FOLLOW-UP QUESTION · ANSWERED

How would you track a Unicode target alphabet?

Count required Unicode code points in Map<Integer,Integer>; slide over a code-point array and adjust its remaining counts. Store UTF-16 offsets when extracting the final substring. A fixed ASCII table is no longer sufficient.

Q021. Reverse an array in place.

✎ SIMPLE ANSWER

Swap mirrored elements until the pointers meet. The input is deliberately mutated; no new result array is required.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[1,2,3,4]	Swap mirrored elements until the pointers meet.	[4,3,2,1]

💡 Each swap fixes two final positions.

JAVA 17 · SOLUTION — Source Q021 · 3 lines

```
static void reverseArray(int[] a) {
  for (int l=0,r=a.length-1;l<r;l++,r--) { int t=a[l]; a[l]=a[r]; a[r]=t; }
}
```

⏱ TIME & SPACE

Time O(n); auxiliary space O(1).

⚠ INTERVIEW PITFALL

Empty and one-element arrays require no swaps.

FOLLOW-UP QUESTION · ANSWERED

How would you reverse only a half-open range?

Accept [fromInclusive, toExclusive). Validate 0 <= from <= to <= length; swap elements at from and to-1, then move inward. Half-open bounds make empty ranges simple and match Java collection conventions.

Q022. Rotate an array right by k positions.

✎ SIMPLE ANSWER

Normalize k, then use three reversals: whole array, first k elements, and the remaining elements. This mutates the array and also supports negative k as a left rotation.

SKETCH THE PATTERN

EXAMPLE / INPUT
[1,2,3,4,5], k=2

→

KEY LOGIC
Normalize k, then use three reversals: whole array, first k elements, and the remaining elements.

→

RESULT
[4,5,1,2,3]

💡 Reversing twice restores the order within each moved block.

JAVA 17 · SOLUTION — Source Q022 · 8 lines

```
static void rotateRight(int[] a, int k) {
  if (a.length==0) return;
  k=Math.floorMod(k,a.length);
  reverseRange(a,0,a.length-1); reverseRange(a,0,k-1); reverseRange(a,k,a.length-1);
}
static void reverseRange(int[] a,int l,int r) {
  while(l<r) { int t=a[l]; a[l++]=a[r]; a[r--]=t; }
}
```

⏱ TIME & SPACE

Time O(n); auxiliary space O(1).

⚠ INTERVIEW PITFALL

Normalize before interpreting k; avoid modulo by zero for empty input.

FOLLOW-UP QUESTION · ANSWERED

How would a cycle-replacement solution work?

Move each element directly to its rotated destination (i+k)%n, following disjoint cycles until all indices are visited. There are gcd(n,k) cycles. Carry one temporary value and carefully mark cycle completion.

Q023. Move zeros to the end while preserving nonzero order.

✎ SIMPLE ANSWER

Write nonzero elements to a forward write pointer, then fill the remainder with zero. Because writes never overtake unread elements, the operation is safe in place.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[0,1,0,3,12]	Write nonzero elements to a forward write pointer, then fill the remainder with zero.	[1,3,12,0,0]

💡 Nonzero encounter order is retained in the compacted prefix.

JAVA 17 · SOLUTION — Source Q023 · 5 lines

```
static void moveZeros(int[] a) {
  int write=0;
  for(int value:a) if(value!=0) a[write++]=value;
  while(write<a.length) a[write++]=0;
}
```

⏱ TIME & SPACE

Time O(n); auxiliary space O(1).

⚠ INTERVIEW PITFALL

All-zero and empty arrays are valid.

FOLLOW-UP QUESTION · ANSWERED

Can you minimize writes when the array is already compact?

Yes. Only write when the read position differs from the next compacted position, and avoid changing already-correct suffix zeros. This can reduce memory writes for arrays that are mostly or fully compact.

Q024. Find the second-largest distinct value.

✎ SIMPLE ANSWER

Track the largest and second-largest values using nullable state rather than a sentinel that could equal valid input. Return absence if fewer than two distinct values exist.

SKETCH THE PATTERN

EXAMPLE / INPUT
[5,5,3,1]

→

KEY LOGIC
Track the largest and second-largest values using nullable state rather than a sentinel that could equal valid input.

→

RESULT
Equal maxima do not create a distinct runner-up.

💡 Equal maxima do not create a distinct runner-up.

JAVA 17 · SOLUTION — Source Q024 · 8 lines

```
static OptionalInt secondLargest(int[] a) {
  Integer first=null, second=null;
  for(int x:a) {
    if(first==null || x>first) { second=first; first=x; }
    else if(x<first && (second==null || x>second)) second=x;
  }
  return second==null ? OptionalInt.empty() : OptionalInt.of(second);
}
```

⏱ TIME & SPACE

Time O(n); auxiliary space O(1).

⚠ INTERVIEW PITFALL

Integer.MIN_VALUE remains a valid answer; duplicates do not count twice.

FOLLOW-UP QUESTION · ANSWERED

How would you find the kth distinct largest value?

Keep a min-heap of k distinct values plus a HashSet to ignore duplicates, or use a sorted TreeSet capped at k. The heap root is the kth distinct largest once k different values exist.

Q025. Merge two sorted arrays.

✎ SIMPLE ANSWER

Advance the pointer with the smaller next value and append the remainder. Allocate a new output so neither input is mutated.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[1,3], [2,3]	Advance the pointer with the smaller next value and append the remainder.	[1,2,3,3]

💡 The next smallest unused value is always at one of the two fronts.

JAVA 17 · SOLUTION — Source Q025 · 7 lines

```
static int[] mergeSorted(int[] a,int[] b) {
  int[] out=new int[a.length+b.length]; int i=0,j=0,k=0;
  while(i<a.length && j<b.length) out[k++]=a[i]<=b[j] ? a[i++] : b[j++];
  while(i<a.length) out[k++]=a[i++];
  while(j<b.length) out[k++]=b[j++];
  return out;
}
```

⏱ TIME & SPACE

Time O(n+m); auxiliary space O(1), excluding O(n+m) output.

⚠ INTERVIEW PITFALL

Inputs must already be sorted; empty inputs work.

FOLLOW-UP QUESTION · ANSWERED

How would you merge into a first array with spare capacity?

Assuming the first array has m valid values and room for n more, write backward from index m+n-1 using the larger of a[m-1] and b[n-1]. This avoids overwriting unread values in the first array.

Q026. Return indices for two sum.

✎ SIMPLE ANSWER

Remember earlier values and their indices. For each element, check whether its complement was seen before inserting this element, preventing reuse of one position.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[2,7,11,15], target=9	Remember earlier values and their indices.	[0,1]

💡 At index one, the remembered 2 supplies the complement of 7.

JAVA 17 · SOLUTION — Source Q026 · 10 lines

```
static int[] twoSum(int[] a,int target) {
  Map<Integer,Integer> seen=new HashMap<>();
  for(int i=0;i<a.length;i++) {
    long need=(long)target-a[i];
    if(need>=Integer.MIN_VALUE && need<=Integer.MAX_VALUE && seen.containsKey((int)need))
      return new int[]{seen.get((int)need),i};
    seen.putIfAbsent(a[i],i);
  }
  return new int[0];
}
```

⏱ TIME & SPACE

Expected time O(n); space O(n).

⚠ INTERVIEW PITFALL

Return an empty array if no pair exists; duplicates can use different indices.

FOLLOW-UP QUESTION · ANSWERED

How would you return every unique value pair?

Store seen values in a set; for each x, look for target-x and add the pair as (min,max) to a set of canonical pairs. Use long for the subtraction and define how duplicates affect whether a pair is valid.

Q027. Count subarrays whose sum equals k.

✎ SIMPLE ANSWER

A subarray sum is a difference of prefix sums. Count previously seen prefixes equal to currentPrefix-k, then record the current prefix.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[1,1,1], k=2	A subarray sum is a difference of prefix sums.	Repeated prefixes represent several valid starting positions, so sto

💡 Repeated prefixes represent several valid starting positions, so store frequencies.

JAVA 17 · SOLUTION — Source Q027 · 9 lines

```
static long subarrayCount(int[] a,long k) {
  Map<Long,Long> counts=new HashMap<>(); counts.put(0L,1L);
  long prefix=0, result=0;
  for(int x:a) {
    prefix+=x; result+=counts.getOrDefault(prefix-k,0L);
    counts.merge(prefix,1L,Long::sum);
  }
  return result;
}
```

⏱ TIME & SPACE

Expected time O(n); space O(n).

⚠ INTERVIEW PITFALL

Works with negative numbers. Assume prefix-k fits long; use checked arithmetic for wider target contracts.

FOLLOW-UP QUESTION · ANSWERED

Why does a positive-only sliding window fail with negative values?

A sliding sum is not monotonic when negative values are allowed: adding a number can decrease the sum and removing one can increase it. Prefix-sum frequencies work for arbitrary signed values.

Q028. Find the longest consecutive-value sequence.

✎ SIMPLE ANSWER

Place distinct values in a set and start counting only at values with no predecessor. Guard MIN_VALUE and MAX_VALUE arithmetic so integer wraparound cannot create false adjacency.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[100,4,200,1,3,2]	Place distinct values in a set and start counting only at values with no predecessor.	Only 1 starts the run 1,2,3,4; internal values do not rescan it.

💡 Only 1 starts the run 1,2,3,4; internal values do not rescan it.

JAVA 17 · SOLUTION — Source Q028 · 10 lines

```
static int longestConsecutive(int[] a) {
  Set<Integer> values=new HashSet<>(); for(int x:a) values.add(x);
  int best=0;
  for(int x:values) if(x==Integer.MIN_VALUE || !values.contains(x-1)) {
    int end=x, length=1;
    while(end!=Integer.MAX_VALUE && values.contains(end+1)) { end++; length++; }
    best=Math.max(best,length);
  }
  return best;
}
```

⏱ TIME & SPACE

Expected time O(n); space O(n).

⚠ INTERVIEW PITFALL

Duplicates are ignored; integer endpoints must not wrap.

FOLLOW-UP QUESTION · ANSWERED

How would sorting trade extra time for a simpler representation?

Sort the values in O(n log n) time, then make one pass counting consecutive distinct numbers. It needs only constant extra state if mutation is allowed; the HashSet method uses expected O(n) time but O(n) memory.

Q029. Find a majority element, if one exists.

✎ SIMPLE ANSWER

Boyer-Moore cancels different values to obtain a candidate. A second pass verifies strict frequency greater than n/2 because existence is not assumed here.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[2,2,1,2]	Boyer-Moore cancels different values to obtain a candidate.	2; [1,2] -> absent

💡 Pair cancellation preserves an existing strict-majority candidate.

JAVA 17 · SOLUTION — Source Q029 · 9 lines

```
static OptionalInt majority(int[] a) {
  int candidate=0, balance=0;
  for(int x:a) {
    if(balance==0) candidate=x;
    balance+=x==candidate ? 1 : -1;
  }
  int count=0; for(int x:a) if(x==candidate) count++;
  return count>a.length/2 ? OptionalInt.of(candidate) : OptionalInt.empty();
}
```

⏱ TIME & SPACE

Time O(n); space O(1).

⚠ INTERVIEW PITFALL

Candidate selection alone does not establish existence; empty input returns absent.

FOLLOW-UP QUESTION · ANSWERED

How would you find values occurring more than n/3 times?

Boyer-Moore generalizes to two candidates, because no more than two values can occur over n/3 times. Use a second pass to verify both, since candidate selection alone does not guarantee either threshold.

Q030. Intersect arrays while preserving multiplicities.

✎ SIMPLE ANSWER

Count the first array and consume counts while scanning the second. Output order follows the second array; each matched occurrence reduces available frequency.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[1,2,2,1], [2,2,2]	Count the first array and consume counts while scanning the second.	[2,2]

💡 A count of two permits exactly two matches, not three.

JAVA 17 · SOLUTION — Source Q030 · 8 lines

```
static List<Integer> intersectionCounts(int[] a,int[] b) {
  Map<Integer,Integer> counts=new HashMap<>(); for(int x:a) counts.merge(x,1,Integer::sum);
  List<Integer> out=new ArrayList<>();
  for(int x:b) if(counts.getOrDefault(x,0)>0) {
    out.add(x); counts.put(x,counts.get(x)-1);
  }
  return out;
}
```

⏱ TIME & SPACE

Expected time O(n+m); space O(u), excluding output.

⚠ INTERVIEW PITFALL

This is multiset intersection, not distinct set intersection.

FOLLOW-UP QUESTION · ANSWERED

How would sorted arrays permit a constant-space scan?

With both arrays sorted, compare front values and advance the smaller pointer. On equality, emit one occurrence and advance both. It runs in O(n+m) time and O(1) extra space excluding the output.

Q031. Find a two-sum pair in a sorted array.

✎ SIMPLE ANSWER

Use pointers at both ends. A sum below target requires a larger left value; a sum above target requires a smaller right value.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[1,2,4,7], target=6	Use pointers at both ends.	[1,2] indices

💡 Sorted order justifies discarding one endpoint at a time.

JAVA 17 · SOLUTION — Source Q031 · 9 lines

```
static int[] sortedTwoSum(int[] a,long target) {
  int l=0,r=a.length-1;
  while(l<r) {
    long sum=(long)a[l]+a[r];
    if(sum==target) return new int[]{l,r};
    if(sum<target) l++; else r--;
  }
  return new int[0];
}
```

⏱ TIME & SPACE

Time O(n); space O(1), excluding returned pair.

⚠ INTERVIEW PITFALL

Ascending input is required; one position cannot be reused.

FOLLOW-UP QUESTION · ANSWERED

Why does this pointer movement fail on unsorted input?

The two-pointer proof relies on ascending order: moving the left pointer always increases or keeps the value, and moving the right always decreases or keeps it. Those guarantees disappear on unsorted data.

Q032. Find all unique triples summing to zero.

✎ SIMPLE ANSWER

Sort a copy, fix each first value and solve the remaining pair with two pointers. Skip duplicates at the first value and after matches.

SKETCH THE PATTERN

EXAMPLE / INPUT
[-1,0,1,2,-1,-4]

→

KEY LOGIC
Sort a copy, fix each first value and solve the remaining pair with two pointers.

→

RESULT
[-1,-1,2], [-1,0,1]

💡 Sorting groups duplicate values and enables monotonic pair movement.

JAVA 17 · SOLUTION Source Q032 · 17 lines

```
static List<List<Integer>> threeSum(int[] input) {
  int[] a=input.clone(); Arrays.sort(a); List<List<Integer>> out=new ArrayList<>();
  for(int i=0;i<a.length-2;i++) {
    if(i>0 && a[i]==a[i-1]) continue;
    int l=i+1,r=a.length-1;
    while(l<r) {
      long sum=(long)a[i]+a[l]+a[r];
      if(sum<0) l++; else if(sum>0) r--;
      else {
        out.add(List.of(a[i],a[l],a[r])); int left=a[l],right=a[r];
        while(l<r && a[l]==left) l++;
        while(l<r && a[r]==right) r--;
      }
    }
  }
  return out;
}
```

⏱ TIME & SPACE

Time O(n squared); space O(n) for copy/sorting plus output.

⚠ INTERVIEW PITFALL

Input remains unchanged; triples are values, not original indices.

FOLLOW-UP QUESTION · ANSWERED

How would a target-sum variant affect pruning?

Replace the zero comparison with the requested target. After sorting, keep one fixed index and move the two pointers based on whether the three-value sum is below or above target. Skip equal values to avoid duplicate triples.

Q033. Compute product of array except self without division.

✎ SIMPLE ANSWER

Store left-prefix products, then multiply by right-suffix products. This handles zeros without special division rules.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[1,2,3,4]	Store left-prefix products, then multiply by right-suffix products.	[24,12,8,6]

💡 At i, the left and right products exclude a[i].

JAVA 17 · SOLUTION — Source Q033 · 8 lines

```
static long[] productExceptSelf(int[] a) {
  long[] out=new long[a.length]; long prefix=1,suffix=1;
  for(int i=0;i<a.length;i++) { out[i]=prefix; prefix=Math.multiplyExact(prefix,a[i]); }
  for(int i=a.length-1;i>=0;i--) {
    out[i]=Math.multiplyExact(out[i],suffix); suffix=Math.multiplyExact(suffix,a[i]);
  }
  return out;
}
```

⏱ TIME & SPACE

Time O(n); auxiliary space O(1), excluding output.

⚠ INTERVIEW PITFALL

All computed prefixes/suffixes must fit long; intermediate overflow can be rejected even if some final outputs fit.

FOLLOW-UP QUESTION · ANSWERED

How would BigInteger broaden the numeric contract?

Maintain BigInteger prefix and suffix products, and return BigInteger[] rather than int[]. This avoids overflow for large products but increases memory and arithmetic cost with the size of the numbers.

Q034. Find the maximum container area.

✎ SIMPLE ANSWER

With nonnegative heights, evaluate the outer pair then move the shorter side. Keeping that shorter side while reducing width cannot improve its best possible area.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[1,8,6,2,5,4,8,3,7]	With nonnegative heights, evaluate the outer pair then move the shorter side.	The shorter boundary limits the current pair.

💡 The shorter boundary limits the current pair.

JAVA 17 · SOLUTION Source Q034 · 9 lines

```
static long maxContainer(int[] height) {
  for(int x:height) if(x<0) throw new IllegalArgumentException();
  int l=0,r=height.length-1; long best=0;
  while(l<r) {
    best=Math.max(best,(long)Math.min(height[l],height[r])*(r-l));
    if(height[l]<=height[r]) l++; else r--;
  }
  return best;
}
```

⏱ TIME & SPACE

Time O(n); space O(1).

⚠ INTERVIEW PITFALL

Fewer than two heights gives zero; negative heights are invalid.

FOLLOW-UP QUESTION · ANSWERED

Why can you safely move either side when heights tie?

Equal end heights mean the current area is limited by both equally. Reducing the width without increasing either limiting height cannot improve the best area, so moving either equal-height side is safe.

Q035. Calculate trapped rainwater.

✎ SIMPLE ANSWER

Build maximum height to the left and right. At each position, water depth is the smaller boundary maximum minus its height.

SKETCH THE PATTERN

EXAMPLE / INPUT
[0,1,0,2,1,0,1,3,2,1,2,1]

→

KEY LOGIC
Build maximum height to the left and right.

→

RESULT
Each column is bounded by its best wall on both sides.

💡 Each column is bounded by its best wall on both sides.

JAVA 17 · SOLUTION — Source Q035 · 9 lines

```
static long trappedWater(int[] h) {
  for(int x:h) if(x<0) throw new IllegalArgumentException();
  int n=h.length; if(n==0) return 0;
  int[] left=new int[n],right=new int[n]; left[0]=h[0]; right[n-1]=h[n-1];
  for(int i=1;i<n;i++) left[i]=Math.max(left[i-1],h[i]);
  for(int i=n-2;i>=0;i--) right[i]=Math.max(right[i+1],h[i]);
  long total=0; for(int i=0;i<n;i++) total+=Math.min(left[i],right[i])-h[i];
  return total;
}
```

⏱ TIME & SPACE

Time O(n); auxiliary space O(n).

⚠ INTERVIEW PITFALL

Nonnegative heights required; use long for total water.

FOLLOW-UP QUESTION · ANSWERED

How do two running maxima reduce space to O(1)?

Keep leftMax and rightMax with pointers at both ends. Whichever side has the lower running maximum has a known water bound, so calculate trapped water there and move that pointer. Only constant extra state is needed.

Q036. Implement binary search.

✎ SIMPLE ANSWER

Maintain an inclusive search interval and discard the half that cannot contain the target. Compute the midpoint from the interval difference to avoid adding two indices.

SKETCH THE PATTERN

EXAMPLE / INPUT
[1,3,5,7], target=5

→

KEY LOGIC
Maintain an inclusive search interval and discard the half that cannot contain the target.

→

RESULT
The target, if present, remains inside the retained interval.

💡 The target, if present, remains inside the retained interval.

JAVA 17 · SOLUTION — Source Q036 · 9 lines

```
static int binarySearch(int[] a,int target) {
  int l=0,r=a.length-1;
  while(l<=r) {
    int mid=l+(r-l)/2;
    if(a[mid]==target) return mid;
    if(a[mid]<target) l=mid+1; else r=mid-1;
  }
  return -1;
}
```

⏱ TIME & SPACE

Time O(log(n+1)); space O(1).

⚠ INTERVIEW PITFALL

Requires sorted input; with duplicates any matching index may be returned.

FOLLOW-UP QUESTION · ANSWERED

How would you require the first matching index?

When a[mid] equals target, store mid and continue searching to the left by setting high=mid-1. Or compute lower_bound(target) and verify that the element at the returned index equals target.

Q037. Find the lower bound of a value.

✎ SIMPLE ANSWER

Search a half-open interval for the first index whose value is at least target. Return n when no such value exists, which is also the insertion position after all elements.

SKETCH THE PATTERN

EXAMPLE / INPUT
[1,2,2,4], target=2

→

KEY LOGIC
Search a half-open interval for the first index whose value is at least target.

→

RESULT
Values before l are too small; a qualifying candidate remains at or

💡 Values before l are too small; a qualifying candidate remains at or before r.

JAVA 17 · SOLUTION — Source Q037 · 8 lines

```
static int lowerBound(int[] a,int target) {
  int l=0,r=a.length;
  while(l<r) {
    int mid=l+(r-l)/2;
    if(a[mid]<target) l=mid+1; else r=mid;
  }
  return l;
}
```

⏱ TIME & SPACE

Time O(log(n+1)); space O(1).

⚠ INTERVIEW PITFALL

The result can equal length and must be checked before indexing.

FOLLOW-UP QUESTION · ANSWERED

How does upper bound differ?

Lower bound finds the first index with a[i] >= x; upper bound finds the first index with a[i] > x. Change the binary-search comparison from < x to <= x while keeping the half-open search invariant.

Q038. Search a rotated sorted array with distinct values.

✎ SIMPLE ANSWER

At least one half is sorted. Check whether target falls within that sorted half; otherwise keep the other half.

SKETCH THE PATTERN

EXAMPLE / INPUT
[4,5,6,7,0,1,2], target=0

→

KEY LOGIC
At least one half is sorted.

→

RESULT
Determine the sorted half before applying range membership.

💡 Determine the sorted half before applying range membership.

JAVA 17 · SOLUTION — Source Q038 · 12 lines

```
static int searchRotated(int[] a,int target) {
  int l=0,r=a.length-1;
  while(l<=r) {
    int m=l+(r-l)/2; if(a[m]==target) return m;
    if(a[l]<=a[m]) {
      if(a[l]<=target && target<a[m]) r=m-1; else l=m+1;
    } else {
      if(a[m]<target && target<=a[r]) l=m+1; else r=m-1;
    }
  }
  return -1;
}
```

⏱ TIME & SPACE

Time O(log(n+1)); space O(1).

⚠ INTERVIEW PITFALL

Duplicates can make the sorted-half decision ambiguous and worst-case search linear.

FOLLOW-UP QUESTION · ANSWERED

How would you handle equal left, middle and right values?

If a[left], a[mid], and a[right] are equal, neither half is provably sorted from those values. Shrink left++ and right--, accepting worst-case O(n) time on many duplicates.

Q039. Find floor square root of a nonnegative int.

✎ SIMPLE ANSWER

Binary-search the largest candidate whose square is no greater than n. Compare using long multiplication so int overflow does not invalidate the predicate.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Find floor square root of a nonnegative int.	Binary-search the largest candidate whose square is no greater than n.	2; 2147483647 -> 46340

💡 The square predicate is monotonic on nonnegative candidates.

JAVA 17 · SOLUTION — Source Q039 · 10 lines

```
static int integerSqrt(int n) {
  if(n<0) throw new IllegalArgumentException();
  int l=0,r=n,answer=0;
  while(l<=r) {
    int m=l+(r-l)/2;
    if((long)m*m<=n) { answer=m; if(m==Integer.MAX_VALUE) break; l=m+1; }
    else r=m-1;
  }
  return answer;
}
```

⏱ TIME & SPACE

Time O(log(n+1)); space O(1).

⚠ INTERVIEW PITFALL

Negative input is rejected; zero returns zero.

FOLLOW-UP QUESTION · ANSWERED

How would the comparison change for a long input?

Keep lo, hi and mid as long, and compare mid <= n/mid when mid > 0. This avoids computing mid*mid where it could overflow signed long. Treat zero separately.

Q040. Find the first and last occurrence in a sorted array.

✎ SIMPLE ANSWER

Find the lower bound, verify equality, then binary-search the first value greater than target. Avoid target+1, which overflows when target is Integer.MAX_VALUE.

SKETCH THE PATTERN

EXAMPLE / INPUT
[5,7,7,8,8,10], 8

→

KEY LOGIC
Find the lower bound, verify equality, then binary-search the first value greater than target.

→

RESULT
[3,4]

💡 Two boundaries identify the whole equal-value range.

JAVA 17 · SOLUTION — Source Q040 · 7 lines

```
static int[] firstLast(int[] a,int target) {
  int first=lowerBound(a,target);
  if(first==a.length || a[first]!=target) return new int[]{-1,-1};
  int l=first,r=a.length;
  while(l<r) { int m=l+(r-l)/2; if(a[m]<=target) l=m+1; else r=m; }
  return new int[]{first,l-1};
}
```

⏱ TIME & SPACE

Time O(log(n+1)); space O(1).

⚠ INTERVIEW PITFALL

This solution calls the lowerBound method from Q037.

FOLLOW-UP QUESTION · ANSWERED

How would you count occurrences from these bounds?

Count = upperBound(target) - lowerBound(target). If the count is zero, the target does not occur. Both bounds are O(log n) with sorted input.

Q041. Implement stable merge sort.

✎ SIMPLE ANSWER

Recursively sort halves and merge into reusable temporary storage. Choose the left value on ties to preserve stability.

SKETCH THE PATTERN

EXAMPLE / INPUT
[4,1,3,2]

→

KEY LOGIC
Recursively sort halves and merge into reusable temporary storage.

→

RESULT
[1,2,3,4]

💡 Sorted halves merge in linear work at each recursion level.

JAVA 17 · SOLUTION — Source Q041 · 9 lines

```
static void mergeSort(int[] a) { mergeSortRange(a,new int[a.length],0,a.length); }
static void mergeSortRange(int[] a,int[] temp,int l,int r) {
  if(r-l<2) return; int m=l+(r-l)/2;
  mergeSortRange(a,temp,l,m); mergeSortRange(a,temp,m,r);
  int i=l,j=m,k=l;
  while(i<m && j<r) temp[k++]=a[i]<=a[j] ? a[i++] : a[j++];
  while(i<m) temp[k++]=a[i++]; while(j<r) temp[k++]=a[j++];
  System.arraycopy(temp,l,a,l,r-l);
}
```

⏱ TIME & SPACE

Time O(n log(n+1)); auxiliary space O(n), including O(log n) recursion.

⚠ INTERVIEW PITFALL

Stable comparison matters when adapting to records with equal keys.

FOLLOW-UP QUESTION · ANSWERED

When does iterative bottom-up merge sort help?

Bottom-up merge sort avoids recursive stack frames and merges runs of width 1, 2, 4, 8, and so on. It is useful for predictable iterative control while retaining O(n log n) time and O(n) merge storage.

Q042. Implement in-place quicksort.

✎ SIMPLE ANSWER

Partition around a selected pivot using two inward scans, then recurse on remaining partitions. This middle-pivot version is simple but does not guarantee balanced partitions or logarithmic stack depth.

SKETCH THE PATTERN

EXAMPLE / INPUT
[3,1,2,3]

→

KEY LOGIC
Partition around a selected pivot using two inward scans, then recurse on remaining partitions.

→

RESULT
[1,2,3,3]

💡 The pivot value stops each scan; exchanges place values on appropriate sides.

JAVA 17 · SOLUTION — Source Q042 · 9 lines

```
static void quickSort(int[] a) { quickSortRange(a,0,a.length-1); }
static void quickSortRange(int[] a,int l,int r) {
  if(l>=r) return; int i=l,j=r,pivot=a[l+(r-l)/2];
  while(i<=j) {
    while(a[i]<pivot) i++; while(a[j]>pivot) j--;
    if(i<=j) { int t=a[i]; a[i++]=a[j]; a[j--]=t; }
  }
  if(l<j) quickSortRange(a,l,j); if(i<r) quickSortRange(a,i,r);
}
```

⏱ TIME & SPACE

Average O(n log n) under suitable partitions; worst O(n squared). Stack worst O(n).

⚠ INTERVIEW PITFALL

In-place is not constant total space when recursion is included; stability is not preserved.

FOLLOW-UP QUESTION · ANSWERED

How do randomized pivots or introsort improve robustness?

Randomized pivots reduce the chance of repeatedly choosing a bad pivot on patterned input. Introsort switches to heapsort when recursion becomes too deep, preserving O(n log n) worst-case time while keeping fast average quicksort behavior.

Q043. Sort an array containing only 0, 1 and 2.

✎ SIMPLE ANSWER

Maintain a zero prefix, a two suffix and an unknown region. Inspect the current element; after swapping a two from the end, recheck the newly received value.

SKETCH THE PATTERN

EXAMPLE / INPUT
[2,0,2,1,1,0]

→

KEY LOGIC
Maintain a zero prefix, a two suffix and an unknown region.

→

RESULT
[0,0,1,1,2,2]

💡 Known regions grow until no unclassified position remains.

JAVA 17 · SOLUTION — Source Q043 · 9 lines

```
static void sort012(int[] a) {
  for(int x:a) if(x<0 || x>2) throw new IllegalArgumentException();
  int low=0,mid=0,high=a.length-1;
  while(mid<=high) {
    if(a[mid]==0) { int t=a[low]; a[low++]=a[mid]; a[mid++]=t; }
    else if(a[mid]==1) mid++;
    else { int t=a[high]; a[high--]=a[mid]; a[mid]=t; }
  }
}
```

⏱ TIME & SPACE

Time O(n); auxiliary space O(1).

⚠ INTERVIEW PITFALL

The initial validation avoids partially mutating invalid input.

FOLLOW-UP QUESTION · ANSWERED

What are the exact invariants of the three regions?

Maintain a[0..low) as zeros, a[low..mid) as ones, a[mid..high] as unknown, and a(high..n) as twos. Each operation shrinks the unknown region while keeping the other zones correct.

Q044. Find the kth largest value with duplicates counted.

✎ SIMPLE ANSWER

Keep the k largest seen values in a min-heap. When the heap grows beyond k, remove its smallest; its root is the current kth largest after all input values.

SKETCH THE PATTERN

EXAMPLE / INPUT
[3,2,1,5,6,4], k=2

→

KEY LOGIC
Keep the k largest seen values in a min-heap.

→

RESULT
Every discarded element is outside the retained top k positions.

💡 Every discarded element is outside the retained top k positions.

JAVA 17 · SOLUTION Source Q044 · 6 lines

```
static int kthLargest(int[] a,int k) {
  if(k<1 || k>a.length) throw new IllegalArgumentException();
  PriorityQueue<Integer> heap=new PriorityQueue<>();
  for(int x:a) { heap.offer(x); if(heap.size()>k) heap.poll(); }
  return heap.element();
}
```

⏱ TIME & SPACE

Time O(n log(k+1)); space O(k).

⚠ INTERVIEW PITFALL

Duplicate values occupy separate ranked positions; k is one-based.

FOLLOW-UP QUESTION · ANSWERED

How does quickselect compare in expected time and mutation?

Quickselect partitions around a pivot and continues only in the partition containing rank k. It has expected O(n) time and O(1) extra partition space, but typically changes array order and can be $O(n^2)$ with poor pivots.

Q045. Merge overlapping closed intervals.

✎ SIMPLE ANSWER

Copy and sort intervals by start. Extend the current interval while the next start is within its closed end; otherwise start a new output interval.

SKETCH THE PATTERN

EXAMPLE / INPUT
[1,3],[2,6],[8,10]

→

KEY LOGIC
Copy and sort intervals by start.

→

RESULT
[1,6],[8,10]

💡 Sorted starts ensure only the last merged interval can overlap the next.

JAVA 17 · SOLUTION — Source Q045 · 13 lines

```
static int[][] mergeIntervals(int[][] input) {
  int[][] a=new int[input.length][2];
  for(int i=0;i<a.length;i++) {
    if(input[i].length!=2 || input[i][0]>input[i][1]) throw new IllegalArgumentException();
    a[i]=input[i].clone();
  }
  Arrays.sort(a,Comparator.comparingInt(x->x[0])); List<int[]> out=new ArrayList<>();
  for(int[] interval:a) {
    if(out.isEmpty() || interval[0]>out.get(out.size()-1)[1]) out.add(interval);
    else { int[] last=out.get(out.size()-1); last[1]=Math.max(last[1],interval[1]); }
  }
  return out.toArray(int[][]::new);
}
```

⏱ TIME & SPACE

Time O(n log(n+1)); auxiliary space O(n), excluding output.

⚠ INTERVIEW PITFALL

Closed versus half-open intervals change whether touching counts as overlap.

FOLLOW-UP QUESTION · ANSWERED

How would you compute meeting-room count instead?

Sort intervals by start time; push each new meeting's end time into a min-heap, removing end times that finish before the next start. The maximum concurrent heap size is the required number of rooms. Define whether touching intervals overlap.

Q046. Traverse a rectangular matrix in spiral order.

✎ SIMPLE ANSWER

Track top, bottom, left and right boundaries. Visit each border, then shrink it.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[[1,2,3],[4,5,6]]	Track top, bottom, left and right boundaries.	[1,2,3,6,5,4]

💡 Each pass consumes an outer border that will not be revisited.

JAVA 17 · SOLUTION — Source Q046 · 11 lines

```
static List<Integer> spiral(int[][] a) {
  int cols=columns(a),top=0,bottom=a.length-1,left=0,right=cols-1;
  List<Integer> out=new ArrayList<>();
  while(top<=bottom && left<=right) {
    for(int c=left;c<=right;c++) out.add(a[top][c]); top++;
    for(int r=top;r<=bottom;r++) out.add(a[r][right]); right--;
    if(top<=bottom) { for(int c=right;c>=left;c--) out.add(a[bottom][c]); bottom--; }
    if(left<=right) { for(int r=bottom;r>=top;r--) out.add(a[r][left]); left++; }
  }
  return out;
}
```

⏱ TIME & SPACE

Time O(rows*cols); auxiliary space O(1), excluding output.

⚠ INTERVIEW PITFALL

Uses the shared columns validator; zero-column/empty matrices return empty.

FOLLOW-UP QUESTION · ANSWERED

How would you generate an n-by-n spiral matrix?

Create an n-by-n array; keep top, bottom, left and right boundaries. Fill top row, right column, bottom row and left column with increasing numbers, shrinking each boundary after use.

Q047. Rotate a square matrix 90 degrees clockwise in place.

✎ SIMPLE ANSWER

Transpose across the main diagonal, then reverse each row. These two transformations map each element to its clockwise-rotated position.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[[1,2],[3,4]]	Transpose across the main diagonal, then reverse each row.	[[3,1],[4,2]]

💡 Transpose sends (r,c) to (c,r); row reversal completes clockwise rotation.

JAVA 17 · SOLUTION Source Q047 · 7 lines

```
static void rotateMatrix(int[][] a) {
  if(columns(a)!=a.length) throw new IllegalArgumentException();
  for(int r=0;r<a.length;r++) for(int c=r+1;c<a.length;c++) {
    int t=a[r][c]; a[r][c]=a[c][r]; a[c][r]=t;
  }
  for(int[] row:a) reverseArray(row);
}
```

⏱ TIME & SPACE

Time O(n squared); auxiliary space O(1).

⚠ INTERVIEW PITFALL

Calls reverseArray; rectangular nonsquare input is rejected.

FOLLOW-UP QUESTION · ANSWERED

How would counterclockwise rotation change the second operation?

Transpose then reverse column order for a counterclockwise turn, or reverse each row and then transpose. For clockwise rotation the usual sequence is transpose then reverse each row.

Q048. Search a globally sorted rectangular matrix.

✎ SIMPLE ANSWER

Treat rows as one ascending sequence and binary-search a long flattened index. Map each midpoint back with division and remainder.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[[1,3,5],[7,9,11]], target=9	Treat rows as one ascending sequence and binary-search a long flattened index.	true

💡 Flattening preserves ascending order only under the stronger global row-order contract.

JAVA 17 · SOLUTION — Source Q048 · 10 lines

```
static boolean searchMatrix(int[][] a,int target) {
  int cols=columns(a); if(a.length==0 || cols==0) return false;
  long l=0,r=(long)a.length*cols-1;
  while(l<=r) {
    long m=l+(r-l)/2; int value=a[(int)(m/cols)][(int)(m%cols)];
    if(value==target) return true;
    if(value<target) l=m+1; else r=m-1;
  }
  return false;
}
```

⏱ TIME & SPACE

Search O(log(rows*cols)); shape validation O(rows); space O(1).

⚠ INTERVIEW PITFALL

Rows/columns each sorted alone are not sufficient for this algorithm.

FOLLOW-UP QUESTION · ANSWERED

How would staircase search handle only row-and-column sorting?

Start at the top-right cell. If the value is too large, move left; if too small, move down. Row/column ordering justifies discarding one row or column each step; time is O(rows+cols).

Q049. Set every row and column containing a zero to zero.

✎ SIMPLE ANSWER

First mark affected rows and columns using separate boolean arrays, then write zeros in a second pass. Separating observation from mutation prevents newly written zeros from causing extra propagation.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[[1,0],[3,4]]	First mark affected rows and columns using separate boolean arrays, then write zeros in a second pass.	[[0,0],[3,0]]

💡 Only zeros from the original matrix determine markers.

JAVA 17 · SOLUTION — Source Q049 · 7 lines

```
static void zeroMatrix(int[][] a) {
  int cols=columns(a); boolean[] rows=new boolean[a.length],columns=new boolean[cols];
  for(int r=0;r<a.length;r++) for(int c=0;c<cols;c++)
    if(a[r][c]==0) { rows[r]=true; columns[c]=true; }
  for(int r=0;r<a.length;r++) for(int c=0;c<cols;c++)
    if(rows[r] || columns[c]) a[r][c]=0;
}
```

⏱ TIME & SPACE

Time O(rows*cols + rows); auxiliary space O(rows+cols).

⚠ INTERVIEW PITFALL

Mutates input; a zero-width shape still passes validation.

FOLLOW-UP QUESTION · ANSWERED

How can the first row/column serve as O(1)-space markers?

Use the first row and first column to record whether any other cell in that row/column contains zero. Save two flags for their original zero state; then zero marked interiors and finally restore boundary rows/columns.

Q050. Flood-fill a four-connected region.

✎ SIMPLE ANSWER

Use a queue and recolor a cell when enqueuing it, which marks it visited. If the replacement equals the original color, return immediately.

SKETCH THE PATTERN

EXAMPLE / INPUT
[[1,1],[1,0]], start=(0,0), color=2

→

KEY LOGIC
Use a queue and recolor a cell when enqueuing it, which marks it visited.

→

RESULT
[[2,2],[2,0]]

💡 Marking on enqueue prevents duplicate queue entries for one cell.

JAVA 17 · SOLUTION — Source Q050 · 16 lines

```
static void floodFill(int[][] a,int row,int col,int color) {
  int cols=columns(a);
  if(row<0 || row>=a.length || col<0 || col>=cols) throw new IllegalArgumentException();
  int old=a[row][col]; if(old==color) return;
  Deque<int[]> queue=new ArrayDeque<>(); queue.add(new int[]{row,col}); a[row][col]=color;
  int[][] dirs={{1,0},{-1,0},{0,1},{0,-1}};
  while(!queue.isEmpty()) {
    int[] cell=queue.remove();
    for(int[] d:dirs) {
      int r=cell[0]+d[0],c=cell[1]+d[1];
      if(r>=0 && r<a.length && c>=0 && c<cols && a[r][c]==old) {
        a[r][c]=color; queue.add(new int[]{r,c});
      }
    }
  }
}
```

⏱ TIME & SPACE

Time O(rows+visited cells); worst O(rows*cols+rows); queue space O(visited cells).

⚠ INTERVIEW PITFALL

Diagonal cells are not adjacent; invalid starting coordinates are rejected.

FOLLOW-UP QUESTION · ANSWERED

When might recursion overflow the stack?

Recursive flood fill uses one call per path step and can overflow the JVM stack on a large connected grid. Iterative BFS or DFS with a queue/stack stores work on the heap and avoids deep recursive calls.

Q051. Validate balanced brackets.

✎ SIMPLE ANSWER

Push opening brackets and match each closing bracket with the most recent unmatched opening bracket. This version rejects all characters outside the six bracket symbols.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
"{[()]}"	Push opening brackets and match each closing bracket with the most recent unmatched opening bracket.	true; "([)]" -> false

💡 Last-opened must be first-closed.

JAVA 17 · SOLUTION — Source Q051 · 11 lines

```
static boolean brackets(String text) {
  Deque<Character> stack=new ArrayDeque<>();
  for(char c:text.toCharArray()) {
    if(c=='(' || c=='[' || c=='{') stack.push(c);
    else {
      char expected=c==')' ? '(' : c==']' ? '[' : c=='}' ? '{' : 0;
      if(expected==0 || stack.isEmpty() || stack.pop()!=expected) return false;
    }
  }
  return stack.isEmpty();
}
```

⏱ TIME & SPACE

Time O(n); space O(n), including character copy and stack.

⚠ INTERVIEW PITFALL

Empty input passes; unmatched leftover openings fail.

FOLLOW-UP QUESTION · ANSWERED

How would you allow ordinary characters while checking delimiters?

Ignore symbols that are not one of the selected opening/closing delimiters. Push recognized openings and match recognized closings. State whether text inside quotes or escaped delimiters should be ignored separately.

Q052. Find the next greater value to the right.

✎ SIMPLE ANSWER

Scan from right to left, removing candidates no larger than the current value. The stack then exposes the nearest surviving greater value.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[2,1,2,4,3]	Scan from right to left, removing candidates no larger than the current value.	[4,2,4,absent,absent]

💡 Smaller/equal candidates are blocked by the current closer value for earlier positions.

JAVA 17 · SOLUTION — Source Q052 · 9 lines

```
static OptionalInt[] nextGreater(int[] a) {
  OptionalInt[] out=new OptionalInt[a.length]; Deque<Integer> stack=new ArrayDeque<>();
  for(int i=a.length-1;i>=0;i--) {
    while(!stack.isEmpty() && stack.peek()<=a[i]) stack.pop();
    out[i]=stack.isEmpty() ? OptionalInt.empty() : OptionalInt.of(stack.peek());
    stack.push(a[i]);
  }
  return out;
}
```

⏱ TIME & SPACE

Time O(n), each value pushed/popped once; space O(n).

⚠ INTERVIEW PITFALL

The relation is strictly greater, not greater-or-equal.

FOLLOW-UP QUESTION · ANSWERED

How would circular next-greater search use a doubled scan?

Process indices from 2n-1 down to zero, using i % n to map into the original array. Maintain a decreasing stack; fill answers only in the final n relevant visits, so each value can see items across the wraparound.

Q053. Find days until a warmer temperature.

✎ SIMPLE ANSWER

Maintain indices whose next warmer day is unresolved. When a warmer reading arrives, pop colder indices and record their distance to the current index.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[73,74,75,71,69,72,76,73]	Maintain indices whose next warmer day is unresolved.	[1,1,4,2,1,1,0,0]

💡 Each popped day receives its first strictly warmer day.

JAVA 17 · SOLUTION — Source Q053 · 10 lines

```
static int[] warmerDays(int[] temperatures) {
  int[] days=new int[temperatures.length]; Deque<Integer> stack=new ArrayDeque<>();
  for(int i=0;i<temperatures.length;i++) {
    while(!stack.isEmpty() && temperatures[i]>temperatures[stack.peek()]) {
      int previous=stack.pop(); days[previous]=i-previous;
    }
    stack.push(i);
  }
  return days;
}
```

⏱ TIME & SPACE

Time O(n); space O(n), excluding output.

⚠ INTERVIEW PITFALL

Equal temperatures do not resolve a wait; no warmer day yields zero.

FOLLOW-UP QUESTION · ANSWERED

Could a bounded temperature alphabet support another approach?

If temperatures belong to a small fixed integer range, keep the most recent future index for each possible temperature while scanning backward; find the closest warmer recorded temperature. Cost depends on alphabet size rather than stack operations.

Q054. Design a stack with constant-time minimum.

✎ SIMPLE ANSWER

Maintain a value stack and a minimum stack. Push a minimum whenever the new value is no larger than the current minimum; pop that minimum when its matching value is removed.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
push 2,1,1; pop	Maintain a value stack and a minimum stack.	minimum still 1

💡 Equal minima need one marker per pushed occurrence.

JAVA 17 · SOLUTION — Source Q054 · 6 lines

```
static class MinStack {
  private final Deque<Integer> values=new ArrayDeque<>(), minima=new ArrayDeque<>();
  void push(int x) { values.push(x); if(minima.isEmpty() || x<=minima.peek()) minima.push(x); }
  int pop() { int x=values.pop(); if(x==minima.peek()) minima.pop(); return x; }
  int min() { return minima.element(); }
}
```

⏱ TIME & SPACE

Each operation O(1); total storage O(n).

⚠ INTERVIEW PITFALL

Empty pop/min throws NoSuchElementException; this class is not thread-safe.

FOLLOW-UP QUESTION · ANSWERED

How could storing a minimum with each entry simplify the invariant?

Store each stack entry as (value, minimumSoFar). push computes min(value, previousMin), pop removes both, and getMin reads the top's saved minimum. All operations are O(1), with O(n) storage.

Q055. Evaluate reverse Polish notation.

✎ SIMPLE ANSWER

Push numbers; for each operator pop the right operand and then the left operand. Apply checked int arithmetic for add/subtract/multiply and explicitly handle division overflow.

SKETCH THE PATTERN

EXAMPLE / INPUT
["2","1","+","3","*"]

→

KEY LOGIC
Push numbers; for each operator pop the right operand and then the left operand.

→

RESULT
For 5 2 -, operand order gives 5-2, not 2-5.

💡 For 5 2 -, operand order gives 5-2, not 2-5.

JAVA 17 · SOLUTION — Source Q055 · 20 lines

```
static int rpn(String[] tokens) {
  Deque<Integer> stack=new ArrayDeque<>();
  for(String token:tokens) {
    if(Set.of("+","-","*","/").contains(token)) {
      if(stack.size()<2) throw new IllegalArgumentException();
      int b=stack.pop(),a=stack.pop(),r;
      switch(token) {
        case "+": r=Math.addExact(a,b); break;
        case "-": r=Math.subtractExact(a,b); break;
        case "*": r=Math.multiplyExact(a,b); break;
        default:
          if(a==Integer.MIN_VALUE && b==-1) throw new ArithmeticException("overflow");
          r=a/b;
      }
      stack.push(r);
    } else stack.push(Integer.parseInt(token));
  }
  if(stack.size()!=1) throw new IllegalArgumentException();
  return stack.pop();
}
```

⏱ TIME & SPACE

Time O(total token characters); space O(token count).

⚠ INTERVIEW PITFALL

Division truncates toward zero and division by zero throws; malformed numbers reject.

FOLLOW-UP QUESTION · ANSWERED

How would infix parsing introduce precedence?

Convert infix tokens to postfix with an operator stack, or use a precedence-aware parser. Multiplication binds more tightly than addition; parentheses override the default order and unary operators need their own rules.

Q056. Find the k most frequent integers with deterministic ties.

✎ SIMPLE ANSWER

Count frequencies, then retain the best k keys with a worst-first heap. Rank higher frequency first and lower numeric value first on ties.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[1,1,2,2,3], k=2	Count frequencies, then retain the best k keys with a worst-first heap.	[1,2]

💡 The heap discards the lowest frequency, breaking a tie against the larger value.

JAVA 17 · SOLUTION Source Q056 · 9 lines

```
static List<Integer> topFrequent(int[] a,int k) {
  if(k<0) throw new IllegalArgumentException();
  Map<Integer,Integer> freq=new HashMap<>(); for(int x:a) freq.merge(x,1,Integer::sum);
  Comparator<Integer> best=Comparator.comparingInt((Integer x)->freq.get(x))
      .reversed().thenComparingInt(x->x);
  PriorityQueue<Integer> heap=new PriorityQueue<>(best.reversed());
  for(int x:freq.keySet()) { heap.offer(x); if(heap.size()>k) heap.poll(); }
  List<Integer> out=new ArrayList<>(heap); out.sort(best); return out;
}
```

⏱ TIME & SPACE

Expected O(n+u log(k+1)+k log(k+1)); space O(u+k), with k capped by u in output.

⚠ INTERVIEW PITFALL

k=0 returns empty; k above distinct count returns every key.

FOLLOW-UP QUESTION · ANSWERED

How would buckets trade heap work for frequency-indexed storage?

Build a frequency map, then buckets indexed by count (0..n). Walk counts from largest to smallest and collect k values, sorting values within equal-frequency buckets if deterministic tie order is required.

Q057. Merge k sorted lists of integers.

✎ SIMPLE ANSWER

Put the front item of each nonempty list into a min-heap. Pop the smallest, append it, and add the following item from the same list.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[[1,4],[2,3],[]]	Put the front item of each nonempty list into a min-heap.	[1,2,3,4]

💡 At most one pending front per source list participates in selection.

JAVA 17 · SOLUTION Source Q057 · 13 lines

```
static List<Integer> mergeK(List<List<Integer>> lists) {
  record Cursor(int value,int list,int index) {}
  PriorityQueue<Cursor> heap=new PriorityQueue<>(Comparator.comparingInt(Cursor::value));
  for(int i=0;i<lists.size();i++) if(!lists.get(i).isEmpty())
    heap.offer(new Cursor(lists.get(i).get(0),i,0));
  List<Integer> out=new ArrayList<>();
  while(!heap.isEmpty()) {
    Cursor c=heap.poll(); out.add(c.value()); int next=c.index()+1;
    if(next<lists.get(c.list()).size())
      heap.offer(new Cursor(lists.get(c.list()).get(next),c.list(),next));
  }
  return out;
}
```

⏱ TIME & SPACE

O(N log(k+1)) time, O(k) auxiliary space; assumes constant-time list indexing.

⚠ INTERVIEW PITFALL

Use random-access sorted lists; LinkedList indexing would change the complexity.

FOLLOW-UP QUESTION · ANSWERED

How would iterators support arbitrary list implementations?

Keep one Iterator per input list and a heap entry containing the current value and iterator. After polling an entry, advance only that iterator. This avoids expensive random indexing into LinkedList or streaming inputs.

Q058. Maintain the kth largest value in a stream.

✎ SIMPLE ANSWER

Keep a min-heap of at most k values. After each insertion, its minimum is the kth largest once at least k values have arrived.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
k=2; add 5	Keep a min-heap of at most k values.	absent; add 1 -> 1; add 8 -> 5

💡 Old values below the retained top k can never affect a future kth-largest answer.

JAVA 17 · SOLUTION — Source Q058 · 8 lines

```
static class KthStream {
  private final int k; private final PriorityQueue<Integer> heap=new PriorityQueue<>();
  KthStream(int k) { if(k<1) throw new IllegalArgumentException(); this.k=k; }
  OptionalInt add(int value) {
    heap.offer(value); if(heap.size()>k) heap.poll();
    return heap.size()<k ? OptionalInt.empty() : OptionalInt.of(heap.element());
  }
}
```

⏱ TIME & SPACE

Each add O(log(k+1)); space O(k).

⚠ INTERVIEW PITFALL

Duplicates count as separate values; concurrent calls require a separate policy.

FOLLOW-UP QUESTION · ANSWERED

How would a changing k affect discarded history?

A fixed-size heap throws away values below the current kth threshold. If k later increases, those discarded values may be needed, so either retain full history or define a new stream/rebuild contract.

Q059. Design a bounded LRU cache.

✎ SIMPLE ANSWER

Use access-order LinkedHashMap and evict the eldest entry after insertion exceeds capacity. A successful get refreshes recency under this implementation.

SKETCH THE PATTERN

EXAMPLE / INPUT
capacity=2: put A,B; get A; put C

→

KEY LOGIC
Use access-order LinkedHashMap and evict the eldest entry after insertion exceeds capacity.

→

RESULT
B evicted

💡 Access order keeps the least recently used entry at the eldest position.

JAVA 17 · SOLUTION — Source Q059 · 10 lines

```
static class LruCache<K,V> extends LinkedHashMap<K,V> {
  private final int capacity;
  LruCache(int capacity) {
    super(16,0.75f,true);
    if(capacity<1) throw new IllegalArgumentException(); this.capacity=capacity;
  }
  @Override protected boolean removeEldestEntry(Map.Entry<K,V> entry) {
    return size()>capacity;
  }
}
```

⏱ TIME & SPACE

Expected O(1) basic get/put; O(capacity) storage.

⚠ INTERVIEW PITFALL

The cache is not thread-safe, and inherited operations are part of its public surface.

FOLLOW-UP QUESTION · ANSWERED

How would composition give tighter API and null policies?

Wrap LinkedHashMap in a dedicated cache class with get/put methods, capacity validation, eviction and a stated null policy. Encapsulation prevents callers from bypassing the invariants of access order and maximum size.

Q060. Find the maximum in every sliding window.

✎ SIMPLE ANSWER

Maintain a deque of candidate indices with decreasing values. Remove expired indices from the front and dominated values from the back.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[1,3,-1,-3,5,3,6,7], k=3	Maintain a deque of candidate indices with decreasing values.	[3,3,5,5,6,7]

💡 A newer no-smaller value dominates an older candidate and expires later.

JAVA 17 · SOLUTION Source Q060 · 11 lines

```
static int[] windowMax(int[] a,int k) {
  if(k<1 || k>a.length) throw new IllegalArgumentException();
  int[] out=new int[a.length-k+1]; Deque<Integer> deque=new ArrayDeque<>();
  for(int i=0;i<a.length;i++) {
    while(!deque.isEmpty() && deque.peekFirst()<=i-k) deque.removeFirst();
    while(!deque.isEmpty() && a[deque.peekLast()]<=a[i]) deque.removeLast();
    deque.addLast(i);
    if(i>=k-1) out[i-k+1]=a[deque.peekFirst()];
  }
  return out;
}
```

⏱ TIME & SPACE

Time O(n); auxiliary space O(k), excluding output.

⚠ INTERVIEW PITFALL

Require 1<=k<=n; storing indices is necessary for expiry.

FOLLOW-UP QUESTION · ANSWERED

How would a heap compare in time and stale-entry management?

Store pairs (value,index) in a max-heap. Before returning the top, remove entries whose index is outside the current window. This costs O(n log n) with stale-entry bookkeeping, versus O(n) amortized time for a monotonic deque.

Q061. Reverse a singly linked list.

✎ SIMPLE ANSWER

Relink one node at a time, keeping the unprocessed next pointer before overwriting it. Return the new head.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Reverse a singly linked list.	Relink one node at a time, keeping the unprocessed next pointer before overwriting it.	2 -> 3 becomes 3 -> 2 -> 1

💡 The processed prefix points backward; head still starts the untouched suffix.

JAVA 17 · SOLUTION — Source Q061 · 7 lines

```
static ListNode reverseList(ListNode head) {
  ListNode previous=null;
  while(head!=null) {
    ListNode next=head.next; head.next=previous; previous=head; head=next;
  }
  return previous;
}
```

⏱ TIME & SPACE

Time O(n); auxiliary space O(1).

⚠ INTERVIEW PITFALL

Node identity is preserved; aliases observe the changed links.

FOLLOW-UP QUESTION · ANSWERED

How would a recursive version change stack usage?

Recursive reversal processes the tail first and then points next.next back to the current node. It still needs O(n) call-stack space and may overflow on very long lists, whereas the iterative method needs O(1) auxiliary space.

Q062. Find the middle node of a linked list.

✎ SIMPLE ANSWER

Advance slow by one node and fast by two. When fast reaches the end, slow is at the middle.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Find the middle node of a linked list.	Advance slow by one node and fast by two.	2 -> 3 -> 4 returns node 3

💡 Fast traverses about twice the distance of slow.

JAVA 17 · SOLUTION — Source Q062 · 5 lines

```
static ListNode middle(ListNode head) {
  ListNode slow=head,fast=head;
  while(fast!=null && fast.next!=null) { slow=slow.next; fast=fast.next.next; }
  return slow;
}
```

⏱ TIME & SPACE

Time O(n); space O(1).

⚠ INTERVIEW PITFALL

Null returns null; acyclic list required.

FOLLOW-UP QUESTION · ANSWERED

How would you choose the first middle instead?

Start fast at head.next rather than head (after checking head), or stop when fast.next.next is unavailable. For even-length lists this returns the first of the two middle nodes.

Q063. Detect a cycle with Floyd's algorithm.

✎ SIMPLE ANSWER

Move slow and fast pointers at different speeds. A reachable cycle makes them meet; an acyclic chain lets fast reach null.

SKETCH THE PATTERN

EXAMPLE / INPUT
A repeated value is not necessarily a repeated node.

→

KEY LOGIC
Move slow and fast pointers at different speeds.

→

RESULT
A repeated value is not necessarily a repeated node.

💡 Inside a cycle, the relative position changes by one each step.

JAVA 17 · SOLUTION — Source Q063 · 8 lines

```
static boolean hasCycle(ListNode head) {
  ListNode slow=head,fast=head;
  while(fast!=null && fast.next!=null) {
    slow=slow.next; fast=fast.next.next;
    if(slow==fast) return true;
  }
  return false;
}
```

⏱ TIME & SPACE

Time O(n reachable nodes); space O(1).

⚠ INTERVIEW PITFALL

Value equality cannot detect structural cycles.

FOLLOW-UP QUESTION · ANSWERED

How would you find the cycle's entry node?

Once slow and fast meet, reset one pointer to head and move both one step at a time. Their next meeting is the cycle entry, because the path-length differences are multiples of the cycle length.

Q064. Merge two sorted linked lists.

✎ SIMPLE ANSWER

Build the merged chain by linking the smaller front node each time. A dummy node removes a special case for the first choice.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Merge two sorted linked lists.	Build the merged chain by linking the smaller front node each time.	3 and 2 -> 4 become 1 -> 2 -> 3 -> 4

💡 The tail always ends the correctly ordered merged prefix.

JAVA 17 · SOLUTION — Source Q064 · 9 lines

```
static ListNode mergeLists(ListNode a,ListNode b) {
  ListNode dummy=new ListNode(0),tail=dummy;
  while(a!=null && b!=null) {
    if(a.value<=b.value) { tail.next=a; a=a.next; }
    else { tail.next=b; b=b.next; }
    tail=tail.next;
  }
  tail.next=a!=null ? a : b; return dummy.next;
}
```

⏱ TIME & SPACE

Time O(n+m); auxiliary space O(1).

⚠ INTERVIEW PITFALL

Shared input nodes violate this mutation contract and can produce bad links.

FOLLOW-UP QUESTION · ANSWERED

How would you preserve both input chains?

Instead of relinking input nodes, allocate new nodes containing the selected values and build a new chain. This uses O(n+m) additional space but leaves the original lists intact.

Q065. Remove the nth node from the end.

✎ SIMPLE ANSWER

Use a dummy node and keep a gap of n nodes between fast and slow. When fast reaches the tail, slow precedes the node to remove.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Remove the nth node from the end.	Use a dummy node and keep a gap of n nodes between fast and slow.	2 -> 3, n=2 becomes 1 -> 3

💡 The gap places slow exactly one link before the target.

JAVA 17 · SOLUTION — Source Q065 · 7 lines

```
static ListNode removeNth(ListNode head,int n) {
  if(n<1) throw new IllegalArgumentException();
  ListNode dummy=new ListNode(0); dummy.next=head; ListNode fast=dummy,slow=dummy;
  for(int i=0;i<n;i++) { fast=fast.next; if(fast==null) throw new IllegalArgumentException(); }
  while(fast.next!=null) { fast=fast.next; slow=slow.next; }
  slow.next=slow.next.next; return dummy.next;
}
```

⏱ TIME & SPACE

Time O(n list length); space O(1).

⚠ INTERVIEW PITFALL

Acyclic input; removing the head is handled through dummy.

FOLLOW-UP QUESTION · ANSWERED

How would you report the removed node as well?

Save the removed node before changing the predecessor's next pointer, detach its next link if ownership requires it, and return both the new head and removed node using a record or result object.

Q066. Locate a linked-list cycle entry.

✎ SIMPLE ANSWER

After a slow/fast meeting, reset one pointer to the head. Advancing both one step makes them meet at the entry because their remaining distances align modulo the cycle length.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Locate a linked-list cycle entry.	After a slow/fast meeting, reset one pointer to the head.	2 -> 3 -> 2 returns the original node 2

💡 The second phase locates the structural entry rather than just any cycle node.

JAVA 17 · SOLUTION Source Q066 · 12 lines

```
static ListNode cycleEntry(ListNode head) {
  ListNode slow=head,fast=head;
  while(fast!=null && fast.next!=null) {
    slow=slow.next; fast=fast.next.next;
    if(slow==fast) {
      slow=head;
      while(slow!=fast) { slow=slow.next; fast=fast.next; }
      return slow;
    }
  }
  return null;
}
```

⏱ TIME & SPACE

Time O(n reachable nodes); space O(1).

⚠ INTERVIEW PITFALL

Null means no reachable cycle; compare identity.

FOLLOW-UP QUESTION · ANSWERED

How would you compute cycle length after the meeting?

After Floyd's pointers meet, keep one pointer at the meeting point and advance a second pointer until it returns. The number of steps is the cycle length.

Q067. Find the intersection of two acyclic lists by node identity.

✎ SIMPLE ANSWER

Switch each pointer to the other head when it reaches null. Both traverse equal combined lengths and meet at the shared node or null.

SKETCH THE PATTERN

EXAMPLE / INPUT
A and B share one tail

→

KEY LOGIC
Switch each pointer to the other head when it reaches null.

→

RESULT
return its first shared node

💡 Switching heads compensates for different private-prefix lengths.

JAVA 17 · SOLUTION Source Q067 · 5 lines

```
static ListNode listIntersection(ListNode a,ListNode b) {
  ListNode p=a,q=b;
  while(p!=q) { p=p==null ? b : p.next; q=q==null ? a : q.next; }
  return p;
}
```

⏱ TIME & SPACE

Time O(n+m); space O(1).

⚠ INTERVIEW PITFALL

Equal values in distinct nodes are not an intersection.

FOLLOW-UP QUESTION · ANSWERED

How do cycles change the possible intersection cases?

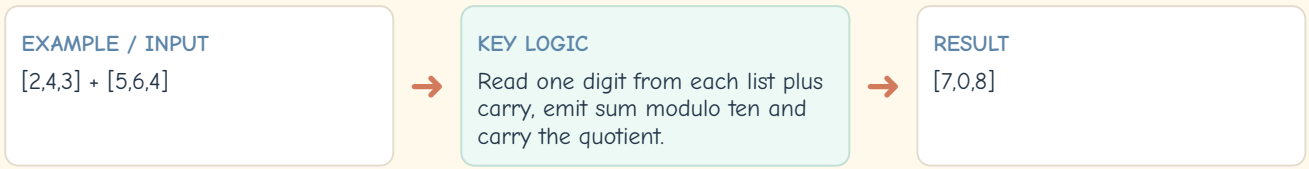
When cycles exist, first detect and locate each entry. A cyclic and acyclic list cannot share a node; two cyclic lists may share an entry or meet at different positions on one cycle. Handle these cases separately.

Q068. Add numbers stored in reverse-order digit lists.

✎ SIMPLE ANSWER

Read one digit from each list plus carry, emit sum modulo ten and carry the quotient. Allocate a fresh result.

SKETCH THE PATTERN



💡 [2,4,3] represents 342; 342+465=807, output [7,0,8].

JAVA 17 · SOLUTION Source Q068 · 10 lines

```
static ListNode addDigits(ListNode a,ListNode b) {
  ListNode dummy=new ListNode(0),tail=dummy; int carry=0;
  while(a!=null || b!=null || carry!=0) {
    int x=a==null ? 0 : a.value,y=b==null ? 0 : b.value;
    if(x<0 || x>9 || y<0 || y>9) throw new IllegalArgumentException();
    int sum=x+y+carry; tail.next=new ListNode(sum%10); tail=tail.next; carry=sum/10;
    if(a!=null) a=a.next; if(b!=null) b=b.next;
  }
  return dummy.next;
}
```

⏱ TIME & SPACE

Time O(max(n,m)); auxiliary space O(1), excluding result nodes.

⚠ INTERVIEW PITFALL

Null plus null returns null under this empty-input convention; acyclic lists required.

FOLLOW-UP QUESTION · ANSWERED

What changes for most-significant-digit-first lists?

For most-significant-digit-first lists, push digits onto two stacks, pop from least significant to most significant with carry, and prepend output nodes. Alternatively reverse copied lists, add, and reverse the result.

Q069. Reverse nodes in groups of k.

✎ SIMPLE ANSWER

First find a complete group; leave a short final group unchanged. Reverse links within the group using the following node as the initial previous pointer, then reconnect the previous group.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Reverse nodes in groups of k.	First find a complete group; leave a short final group unchanged.	2 -> 3 -> 4 -> 5, k=2 -> 2 -> 1 -> 4 -> 3 -> 5

💡 The old first becomes the reversed group tail and the next reconnect point.

JAVA 17 · SOLUTION — Source Q069 · 13 lines

```
static ListNode reverseGroups(ListNode head,int k) {
  if(k<1) throw new IllegalArgumentException();
  ListNode dummy=new ListNode(0); dummy.next=head; ListNode before=dummy;
  while(true) {
    ListNode end=before; for(int i=0;i<k && end!=null;i++) end=end.next;
    if(end==null) return dummy.next;
    ListNode after=end.next,oldFirst=before.next,previous=after,current=oldFirst;
    while(current!=after) {
      ListNode next=current.next; current.next=previous; previous=current; current=next;
    }
    before.next=end; before=oldFirst;
  }
}
```

⏱ TIME & SPACE

Time O(n); auxiliary space O(1).

⚠ INTERVIEW PITFALL

Input is acyclic and mutated; k=1 still terminates and preserves order.

FOLLOW-UP QUESTION · ANSWERED

How would you reverse the final short group too?

Count the available nodes up to k and reverse min(k,remaining) rather than returning when the group is short. Be explicit that this changes the original standard contract and may reverse a one-node tail trivially.

Q070. Check a list palindrome and restore its links.

✎ SIMPLE ANSWER

Find the first-half end, reverse the second half, compare corresponding values, and reverse the second half back before returning. Restoration matters to callers holding aliases to the original list.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Check a list palindrome and restore its links.	Find the first-half end, reverse the second half, compare corresponding values, and reverse the second half back before returning.	2 -> 2 -> 1 -> true, original order retained

💡 The shorter half excludes a center node when the length is odd.

JAVA 17 · SOLUTION Source Q070 · 8 lines

```
static boolean listPalindrome(ListNode head) {
  if(head==null || head.next==null) return true;
  ListNode slow=head,fast=head;
  while(fast.next!=null && fast.next.next!=null) { slow=slow.next; fast=fast.next.next; }
  ListNode reversed=reverseList(slow.next),a=head,b=reversed; boolean same=true;
  while(b!=null) { if(a.value!=b.value) same=false; a=a.next; b=b.next; }
  slow.next=reverseList(reversed); return same;
}
```

⏱ TIME & SPACE

Time O(n); auxiliary space O(1).

⚠ INTERVIEW PITFALL

Acyclic list; it is temporarily mutated, so concurrent observers are outside the contract.

FOLLOW-UP QUESTION · ANSWERED

How would an immutable/nonmutating alternative change space?

Copy values into an array/list and use two pointers, which costs O(n) extra memory but does not change links. A stack for half the values is another O(n) nonmutating option.

Q071. Traverse a binary tree in preorder iteratively.

✎ SIMPLE ANSWER

Push the root, then pop and visit it. Push right before left so the stack visits left next.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Root 1, left 2, right 3	Push the root, then pop and visit it.	[1,2,3]

💡 The stack contains roots of unvisited subtrees in next-visit order.

JAVA 17 · SOLUTION Source Q071 · 9 lines

```
static List<Integer> preorder(TreeNode root) {
  List<Integer> out=new ArrayList<>(); Deque<TreeNode> stack=new ArrayDeque<>();
  if(root!=null) stack.push(root);
  while(!stack.isEmpty()) {
    TreeNode n=stack.pop(); out.add(n.value);
    if(n.right!=null) stack.push(n.right); if(n.left!=null) stack.push(n.left);
  }
  return out;
}
```

⏱ TIME & SPACE

Time O(n); auxiliary space O(height), excluding output.

⚠ INTERVIEW PITFALL

Do not push null into ArrayDeque.

FOLLOW-UP QUESTION · ANSWERED

How would inorder change the traversal invariant?

For inorder, walk left repeatedly while pushing nodes, then pop/visit and move to the right child. Unlike preorder, visiting happens after exhausting the left subtree; the stack stores unfinished ancestors.

Q072. Return binary-tree level order.

✎ SIMPLE ANSWER

Use a queue and process its current size as one level. Enqueue children for the next level, keeping level boundaries explicit.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Root 2 with children 1,3	Use a queue and process its current size as one level.	[[2],[1,3]]

💡 Items added during one level belong to the following level.

JAVA 17 · SOLUTION — Source Q072 · 13 lines

```
static List<List<Integer>> levelOrder(TreeNode root) {
  List<List<Integer>> out=new ArrayList<>(); Deque<TreeNode> queue=new ArrayDeque<>();
  if(root!=null) queue.add(root);
  while(!queue.isEmpty()) {
    int size=queue.size(); List<Integer> level=new ArrayList<>();
    for(int i=0;i<size;i++) {
      TreeNode n=queue.remove(); level.add(n.value);
      if(n.left!=null) queue.add(n.left); if(n.right!=null) queue.add(n.right);
    }
    out.add(level);
  }
  return out;
}
```

⏱ TIME & SPACE

Time O(n); auxiliary space O(maximum width), excluding output.

⚠ INTERVIEW PITFALL

Null root gives no levels.

FOLLOW-UP QUESTION · ANSWERED

How would you alternate the output direction for zigzag traversal?

At each BFS depth, fill a list or deque from left-to-right for even levels and right-to-left for odd levels. Preserve the same queue for discovering children; only reverse each level's result order.

Q073. Compute maximum binary-tree depth.

✎ SIMPLE ANSWER

A null subtree has depth zero; a node's depth is one plus the maximum depth of its children. Return the number of nodes on the longest root-to-leaf path.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Root with two leaf children	A null subtree has depth zero; a node's depth is one plus the maximum depth of its children.	depth 2

💡 The recurrence follows directly from choosing the deeper child path.

JAVA 17 · SOLUTION Source Q073 · 3 lines

```
static int maxDepth(TreeNode root) {
  return root==null ? 0 : 1+Math.max(maxDepth(root.left),maxDepth(root.right));
}
```

⏱ TIME & SPACE

Time O(n); recursion space O(height).

⚠ INTERVIEW PITFALL

A skewed tree can exhaust the Java call stack; use iterative traversal for extreme depth.

FOLLOW-UP QUESTION · ANSWERED

How does minimum depth differ when one child is missing?

At a node with only one child, minimum depth must follow the existing child; a missing child is not a valid root-to-leaf path. If both children exist, return 1 + min(child depths); if none, return 1.

Q074. Find binary-tree diameter in edges.

✎ SIMPLE ANSWER

During one postorder traversal, return subtree height and update the best sum of left and right heights. A longest path may pass through any node, not only the root.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Three-node root with two leaves	During one postorder traversal, return subtree height and update the best sum of left and right heights.	diameter 2

💡 Heights measured in nodes produce a through-node path length in edges.

JAVA 17 · SOLUTION — Source Q074 · 8 lines

```
static int diameter(TreeNode root) {
  int[] best={0}; diameterHeight(root,best); return best[0];
}
static int diameterHeight(TreeNode n,int[] best) {
  if(n==null) return 0;
  int left=diameterHeight(n.left,best),right=diameterHeight(n.right,best);
  best[0]=Math.max(best[0],left+right); return 1+Math.max(left,right);
}
```

⏱ TIME & SPACE

Time O(n); recursion space O(height).

⚠ INTERVIEW PITFALL

Null/one-node trees have diameter zero; define edges versus nodes first.

FOLLOW-UP QUESTION · ANSWERED

How would you return the actual path nodes?

Have DFS return each node's best downward path and update a global best when left-depth + right-depth improves diameter. Retain parent pointers or the chosen endpoints, then reconstruct their connecting path.

Q075. Check whether a binary tree is height-balanced.

✎ SIMPLE ANSWER

Return a subtree height if balanced, otherwise a negative sentinel. Propagate failure immediately, avoiding separate repeated height calculations for every node.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
A three-node one-sided chain	Return a subtree height if balanced, otherwise a negative sentinel.	false

💡 A balanced node requires balanced children and height difference at most one.

JAVA 17 · SOLUTION Source Q075 · 8 lines

```
static boolean balanced(TreeNode root) { return balancedHeight(root)>=0; }
static int balancedHeight(TreeNode n) {
  if(n==null) return 0;
  int left=balancedHeight(n.left); if(left<0) return -1;
  int right=balancedHeight(n.right);
  if(right<0 || Math.abs(left-right)>1) return -1;
  return 1+Math.max(left,right);
}
```

⏱ TIME & SPACE

Time O(n); recursion space O(height).

⚠ INTERVIEW PITFALL

Balanced is not equivalent to complete or perfect.

FOLLOW-UP QUESTION · ANSWERED

Why can a naive per-node depth check become quadratic?

If every node calls a full depth traversal of its descendants, shared subtrees get recomputed repeatedly. On a skewed tree this can take $O(n^2)$. Returning depth and balanced status in one postorder visit uses O(n).

Q076. Validate a strict binary search tree.

✎ SIMPLE ANSWER

Propagate exclusive lower and upper bounds through the whole subtree. Use long bounds so valid int endpoints remain representable.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
A value in the left subtree must be below every relevant ancest…	Propagate exclusive lower and upper bounds through the whole subtree.	A value in the left subtree must be below every relevant ancestor bound.

💡 Checking only each parent/child pair can miss an ancestor-bound violation.

JAVA 17 · SOLUTION Source Q076 · 8 lines

```
static boolean validBST(TreeNode root) {
  return validBounds(root,Long.MIN_VALUE,Long.MAX_VALUE);
}
static boolean validBounds(TreeNode n,long low,long high) {
  if(n==null) return true;
  return n.value>low && n.value<high
      && validBounds(n.left,low,n.value) && validBounds(n.right,n.value,high);
}
```

⏱ TIME & SPACE

Time O(n); recursion space O(height).

⚠ INTERVIEW PITFALL

This is a strict BST with no duplicate keys; null is valid.

FOLLOW-UP QUESTION · ANSWERED

How would an explicit duplicate-placement policy change bounds?

Choose a documented policy such as duplicates only in the right subtree. Then the left range stays strictly less than node and the right range allows equality at its lower bound; track inclusive/exclusive limits carefully.

Q077. Find the kth smallest node value in a BST.

✎ SIMPLE ANSWER

Inorder traversal visits a valid BST in ascending order. Use an explicit stack and return when the one-based visit count reaches k.

SKETCH THE PATTERN

EXAMPLE / INPUT
BST 2 with children 1,3; k=2

→

KEY LOGIC
Inorder traversal visits a valid BST in ascending order.

→

RESULT
The kth inorder visit supplies the rank.

💡 The kth inorder visit supplies the rank.

JAVA 17 · SOLUTION — Source Q077 · 9 lines

```
static int kthSmallest(TreeNode root,int k) {
  if(k<1) throw new IllegalArgumentException();
  Deque<TreeNode> stack=new ArrayDeque<>(); TreeNode current=root;
  while(current!=null || !stack.isEmpty()) {
    while(current!=null) { stack.push(current); current=current.left; }
    current=stack.pop(); if(--k==0) return current.value; current=current.right;
  }
  throw new IllegalArgumentException("k exceeds node count");
}
```

⏱ TIME & SPACE

Time O(height+k) for valid k; worst O(n). Space O(height).

⚠ INTERVIEW PITFALL

Requires a valid BST; duplicate ranks depend on its duplicate policy.

FOLLOW-UP QUESTION · ANSWERED

How could subtree-size metadata support frequent rank queries?

Store the size of each node's subtree. With leftSize known, compare k to leftSize+1 and descend left or right, adjusting k. Balanced tree queries are O(log n), but inserts/deletes must maintain sizes.

Q078. Find the lowest common ancestor of two present BST keys.

✎ SIMPLE ANSWER

If both keys are smaller, descend left; if both are larger, descend right. The first split or matching node is the ancestor.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
BST 2 with keys 1 and 3	If both keys are smaller, descend left; if both are larger, descend right.	root 2

💡 A split means the two paths cannot continue together below that node.

JAVA 17 · SOLUTION — Source Q078 · 9 lines

```
static TreeNode lcaBST(TreeNode root,int a,int b) {
  int low=Math.min(a,b),high=Math.max(a,b);
  while(root!=null) {
    if(high<root.value) root=root.left;
    else if(low>root.value) root=root.right;
    else return root;
  }
  return null;
}
```

⏱ TIME & SPACE

Time O(height); space O(1).

⚠ INTERVIEW PITFALL

Without the presence precondition, a split is not proof both keys exist.

FOLLOW-UP QUESTION · ANSWERED

How would general binary-tree LCA differ?

In a general binary tree there is no key ordering for choosing a side. Search both subtrees recursively and return a node when the two targets split, or store parent links and find their common ancestor.

Q079. Serialize a binary tree with structure preserved.

✎ SIMPLE ANSWER

Write preorder values and an explicit marker for every null child. Values alone do not encode an arbitrary tree's shape.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Tree 2 with children 1,3	Write preorder values and an explicit marker for every null child.	"2,1,#,#,3,#,#,"

💡 Each node and null boundary appears once in preorder.

JAVA 17 · SOLUTION — Source Q079 · 7 lines

```
static String serialize(TreeNode root) {
  StringBuilder out=new StringBuilder(); serializeNode(root,out); return out.toString();
}
static void serializeNode(TreeNode n,StringBuilder out) {
  if(n==null) { out.append("#,"); return; }
  out.append(n.value).append(','); serializeNode(n.left,out); serializeNode(n.right,out);
}
```

⏱ TIME & SPACE

Time O(n + encoded characters); space O(height) recursion plus output.

⚠ INTERVIEW PITFALL

No cycles or shared-subtree graph semantics; deep recursion can overflow.

FOLLOW-UP QUESTION · ANSWERED

What versioning and validation would a durable wire format need?

Use an explicit format version, a defined character encoding, safe delimiters/escaping and limits on depth/node count. Validate integer ranges, token count and malformed streams before constructing a durable tree.

Q080. Deserialize the preorder tree format.

✎ SIMPLE ANSWER

Read one token for each subtree. A # is null; otherwise parse a node and recursively read its left and right subtrees.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
"2,1,#,#,3,#,#,"	Read one token for each subtree.	the original tree

💡 The parser mirrors the serializer's recursive grammar.

JAVA 17 · SOLUTION Source Q080 · 12 lines

```
static TreeNode deserialize(String encoded) {
  String[] tokens=encoded.split(","); int[] position={0};
  TreeNode root=readTree(tokens,position);
  if(position[0]!=tokens.length) throw new IllegalArgumentException("extra tokens");
  return root;
}
static TreeNode readTree(String[] tokens,int[] p) {
  if(p[0]>=tokens.length) throw new IllegalArgumentException("truncated");
  String token=tokens[p[0]++]; if(token.equals("#")) return null;
  TreeNode n=new TreeNode(Integer.parseInt(token));
  n.left=readTree(tokens,p); n.right=readTree(tokens,p); return n;
}
```

⏱ TIME & SPACE

Time O(encoded characters); space O(encoded characters + height), including token array.

⚠ INTERVIEW PITFALL

Java split drops trailing empty tokens; this accepts optional trailing commas, so it is a practice format, not a canonical protocol.

FOLLOW-UP QUESTION · ANSWERED

How would an iterative parser protect against extreme depth?

Use an explicit stack of pending child slots or frames instead of recursive descent. Process each token and attach children iteratively, validating arity and rejecting extra/truncated input; heap-based stack avoids JVM recursion overflow.

Q081. Find shortest unweighted distances with BFS.

✎ SIMPLE ANSWER

Start from the source, mark its distance zero and discover each neighbor once. Queue order processes increasing edge distance.

SKETCH THE PATTERN

EXAMPLE / INPUT
Find shortest unweighted distances with BFS.

→

KEY LOGIC
Start from the source, mark its distance zero and discover each neighbor once.

→

RESULT
1 -> 2 and isolated 3; source 0 -> [0,1,2,-1]

💡 The first discovery uses a shortest edge-count path.

JAVA 17 · SOLUTION — Source Q081 · 12 lines

```
static int[] bfsDistances(List<List<Integer>> graph,int source) {
  if(source<0 || source>=graph.size()) throw new IllegalArgumentException();
  int[] distance=new int[graph.size()]; Arrays.fill(distance,-1);
  Deque<Integer> queue=new ArrayDeque<>(); queue.add(source); distance[source]=0;
  while(!queue.isEmpty()) {
    int u=queue.remove();
    for(int v:graph.get(u)) if(distance[v]==-1) {
      distance[v]=distance[u]+1; queue.add(v);
    }
  }
  return distance;
}
```

⏱ TIME & SPACE

Time O(V+E) for adjacency lists; space O(V).

⚠ INTERVIEW PITFALL

Neighbor indices must be valid; the graph may be directed.

FOLLOW-UP QUESTION · ANSWERED

Why is this insufficient for unequal weighted edges?

BFS minimizes the number of edges, not total weight. With different nonnegative edge costs, Dijkstra finds minimum total weight; with negative edges use a suitable Bellman-Ford-style algorithm.

Q082. Detect a cycle in a directed graph.

✎ SIMPLE ANSWER

Use three DFS states: unseen, currently active, and finished. An edge to an active vertex is a back edge and proves a directed cycle.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Detect a cycle in a directed graph.	Use three DFS states: unseen, currently active, and finished.	1 -> 0 -> cycle

💡 Finished vertices are safe to revisit; active vertices reveal a recursion-path cycle.

JAVA 17 · SOLUTION — Source Q082 · 13 lines

```
static boolean directedCycle(List<List<Integer>> graph) {
  int[] state=new int[graph.size()];
  for(int u=0;u<state.length;u++) if(state[u]==0 && cycleDFS(graph,u,state)) return true;
  return false;
}
static boolean cycleDFS(List<List<Integer>> graph,int u,int[] state) {
  state[u]=1;
  for(int v:graph.get(u)) {
    if(state[v]==1) return true;
    if(state[v]==0 && cycleDFS(graph,v,state)) return true;
  }
  state[u]=2; return false;
}
```

⏱ TIME & SPACE

Time O(V+E); auxiliary space O(V), including recursion.

⚠ INTERVIEW PITFALL

Valid neighbor indices required; a deep graph can exhaust recursion stack.

FOLLOW-UP QUESTION · ANSWERED

How does undirected cycle detection treat the parent edge?

During DFS on an undirected graph, ignore the edge back to the parent vertex because it is not a cycle by itself. Encountering another already visited neighbor indicates a cycle. Parallel-edge graphs need edge-identity handling.

Q083. Produce a topological ordering or reject a cycle.

✎ SIMPLE ANSWER

Count incoming edges and repeatedly remove zero-indegree vertices, reducing their neighbors' counts. If fewer than V vertices are emitted, a cycle prevents a complete order.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Produce a topological ordering or reject a cycle.	Count incoming edges and repeatedly remove zero-indegree vertices, reducing their neighbors' counts.	1 -> 2 -> [0,1,2]

💡 A zero-indegree vertex has no unresolved predecessor.

JAVA 17 · SOLUTION Source Q083 · 13 lines

```
static List<Integer> topological(List<List<Integer>> graph) {
  int[] degree=new int[graph.size()];
  for(List<Integer> edges:graph) for(int v:edges) degree[v]++;
  Deque<Integer> queue=new ArrayDeque<>();
  for(int u=0;u<degree.length;u++) if(degree[u]==0) queue.add(u);
  List<Integer> out=new ArrayList<>();
  while(!queue.isEmpty()) {
    int u=queue.remove(); out.add(u);
    for(int v:graph.get(u)) if(--degree[v]==0) queue.add(v);
  }
  if(out.size()!=graph.size()) throw new IllegalArgumentException("cycle");
  return out;
}
```

⏱ TIME & SPACE

Time O(V+E); space O(V), excluding output.

⚠ INTERVIEW PITFALL

A DAG can have several valid orders; tests should normally verify edge ordering.

FOLLOW-UP QUESTION · ANSWERED

How would a priority queue provide the lexicographically smallest order?

In Kahn's topological algorithm, put every zero-indegree node in a min-heap instead of a FIFO queue. Repeatedly take the smallest available node and decrement neighbors; the result is the lexicographically smallest valid order.

Q084. Find shortest paths with Dijkstra.

✎ SIMPLE ANSWER

For nonnegative weights, expand the smallest tentative distance and relax outgoing edges. This lazy-heap version ignores stale queue entries after a shorter path has replaced them.

SKETCH THE PATTERN

EXAMPLE / INPUT
Find shortest paths with Dijkstra.

→

KEY LOGIC
For nonnegative weights, expand the smallest tentative distance and relax outgoing edges.

→

RESULT
1 weight 2; 0->2 weight 5; 1->2 weight 1 -> [0,2,3]

💡 A stale entry must not redo relaxation for an outdated distance.

JAVA 17 · SOLUTION Source Q084 · 15 lines

```
static long[] dijkstra(List<List<Edge>> graph,int source) {
  if(source<0 || source>=graph.size()) throw new IllegalArgumentException();
  for(List<Edge> edges:graph) for(Edge e:edges) if(e.weight()<0) throw new
IllegalArgumentException();
  long[] dist=new long[graph.size()]; Arrays.fill(dist,Long.MAX_VALUE); dist[source]=0;
  PriorityQueue<PathState> heap=new
PriorityQueue<>(Comparator.comparingLong(PathState::distance));
  heap.add(new PathState(source,0));
  while(!heap.isEmpty()) {
    PathState s=heap.poll(); if(s.distance()!=dist[s.node()]) continue;
    for(Edge e:graph.get(s.node())) {
      long next=s.distance()+e.weight();
      if(next<dist[e.to()]) { dist[e.to()]=next; heap.add(new PathState(e.to(),next)); }
    }
  }
  return dist;
}
```

⏱ TIME & SPACE

Lazy-heap time O(V+E log(E+1)); space O(V+E).

⚠ INTERVIEW PITFALL

Edge/PathState records are shared helpers; valid vertex indices and nonnegative int weights required.

FOLLOW-UP QUESTION · ANSWERED

Which algorithm handles negative weights and detects negative cycles?

Bellman-Ford allows negative edges and detects a reachable negative cycle if any distance can improve after V-1 rounds. Dijkstra's finalized-distance rule does not work with arbitrary negative weights.

Q085. Count four-connected islands in a grid.

✎ SIMPLE ANSWER

When a land cell is found, count one island and flood it to water with a queue. This mutation marks the component visited without a separate visited grid.

SKETCH THE PATTERN

EXAMPLE / INPUT
[[1,0],[0,1]]

→

KEY LOGIC
When a land cell is found, count one island and flood it to water with a queue.

→

RESULT
2 islands

💡 One BFS consumes exactly one connected component.

JAVA 17 · SOLUTION — Source Q085 · 16 lines

```
static int islands(char[][] grid) {
  int cols=columns(grid),count=0; int[][] dirs={{1,0},{-1,0},{0,1},{0,-1}};
  for(int r=0;r<grid.length;r++) for(int c=0;c<cols;c++) if(grid[r][c]=='1') {
    count++; Deque<int[]> queue=new ArrayDeque<>(); queue.add(new int[]{r,c}); grid[r][c]='0';
    while(!queue.isEmpty()) {
      int[] cell=queue.remove();
      for(int[] d:dirs) {
        int nr=cell[0]+d[0],nc=cell[1]+d[1];
        if(nr>=0 && nr<grid.length && nc>=0 && nc<cols && grid[nr][nc]=='1') {
          grid[nr][nc]='0'; queue.add(new int[]{nr,nc});
        }
      }
    }
  }
  return count;
}
```

⏱ TIME & SPACE

Time O(rows*cols+rows); auxiliary queue space O(rows*cols) worst case.

⚠ INTERVIEW PITFALL

Rectangular binary-character grid required; input is consumed/mutated.

FOLLOW-UP QUESTION · ANSWERED

How would union-find support incremental land additions?

On each new land cell, create a singleton set and increase island count. Union with each adjacent land cell; decrement the count whenever two previously separate sets merge. Path compression and union-by-rank make updates efficient.

Q086. Generate distinct permutations, including duplicate input values.

✎ SIMPLE ANSWER

Sort a copy and backtrack over unused positions. At each depth, skip an equal value when its preceding equal position is still unused; that eliminates duplicate choice orders.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[1,1,2]	Sort a copy and backtrack over unused positions.	[1,1,2], [1,2,1], [2,1,1]

💡 Skip same-value alternatives at one decision depth, not every occurrence globally.

JAVA 17 · SOLUTION Source Q086 · 12 lines

```
static List<List<Integer>> permutations(int[] input) {
  int[] a=input.clone(); Arrays.sort(a); List<List<Integer>> out=new ArrayList<>();
  permutationDFS(a,new boolean[a.length],new ArrayList<>(),out); return out;
}
static void permutationDFS(int[] a,boolean[] used,List<Integer> path,List<List<Integer>> out) {
  if(path.size()==a.length) { out.add(new ArrayList<>(path)); return; }
  for(int i=0;i<a.length;i++) {
    if(used[i] || (i>0 && a[i]==a[i-1] && !used[i-1])) continue;
    used[i]=true; path.add(a[i]); permutationDFS(a,used,path,out);
    path.remove(path.size()-1); used[i]=false;
  }
}
```

⏱ TIME & SPACE

Worst O(n*n!) time and output; auxiliary space O(n), plus sorting copy.

⚠ INTERVIEW PITFALL

Empty input has one permutation: the empty list.

FOLLOW-UP QUESTION · ANSWERED

How would you generate permutations lazily rather than store all?

Implement an iterator-like depth-first generator with explicit state or yield one permutation to a callback as soon as a complete assignment is found. Memory can be O(depth) instead of storing factorially many results.

Q087. Generate every subset of distinct values.

✎ SIMPLE ANSWER

Backtrack from a start index and record the current path at every call. Advancing the start prevents choosing earlier positions again.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[1,2]	Backtrack from a start index and record the current path at every call.	[], [1], [1,2], [2]

💡 The path corresponds to positions selected in increasing index order.

JAVA 17 · SOLUTION Source Q087 · 10 lines

```
static List<List<Integer>> subsets(int[] a) {
  Set<Integer> unique=new HashSet<>(); for(int x:a) if(!unique.add(x)) throw new
IllegalArgumentException();
  List<List<Integer>> out=new ArrayList<>(); subsetDFS(a,0,new ArrayList<>(),out); return out;
}
static void subsetDFS(int[] a,int start,List<Integer> path,List<List<Integer>> out) {
  out.add(new ArrayList<>(path));
  for(int i=start;i<a.length;i++) {
    path.add(a[i]); subsetDFS(a,i+1,path,out); path.remove(path.size()-1);
  }
}
```

⏱ TIME & SPACE

Time/output O(n*2^n); auxiliary space O(n).

⚠ INTERVIEW PITFALL

Large n creates exponential output even when recursion is correct.

FOLLOW-UP QUESTION · ANSWERED

How would duplicate values require a sorted skip rule?

Sort first. At each recursion depth, skip value a[i] when i>start and a[i]==a[i-1] so equivalent choices at the same depth are not repeated. This preserves unique subsets without a global result set.

Q088. Find combinations summing to a target with unlimited reuse.

✎ SIMPLE ANSWER

Sort and deduplicate positive candidates. Backtrack with a nondecreasing candidate index; retaining the chosen index allows reuse without producing order permutations.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[2,3,6,7], target=7	Sort and deduplicate positive candidates.	[2,2,3], [7]

💡 Canonical nondecreasing choices avoid emitting [3,2,2] as a second copy.

JAVA 17 · SOLUTION Source Q088 · 12 lines

```
static List<List<Integer>> combinationSum(int[] candidates,int target) {
  if(target<0) throw new IllegalArgumentException();
  for(int x:candidates) if(x<=0) throw new IllegalArgumentException();
  int[] a=Arrays.stream(candidates).distinct().sorted().toArray();
  List<List<Integer>> out=new ArrayList<>(); combineDFS(a,0,target,new ArrayList<>(),out); return
out;
}
static void combineDFS(int[] a,int start,int remaining,List<Integer> path,List<List<Integer>> out)
{
  if(remaining==0) { out.add(new ArrayList<>(path)); return; }
  for(int i=start;i<a.length && a[i]<=remaining;i++) {
    path.add(a[i]); combineDFS(a,i,remaining-a[i],path,out); path.remove(path.size()-1);
  }
}
```

⏱ TIME & SPACE

Output-sensitive exponential search; depth at most target/minCandidate; preprocessing O(n log n).

⚠ INTERVIEW PITFALL

Zero/negative candidates would break the decreasing-target argument.

FOLLOW-UP QUESTION · ANSWERED

How does single-use combination search change the recursive index?

With unlimited reuse, recurse using the same candidate index. For single-use combinations, recurse from i+1 after selecting a value. Sorting plus skipping equal candidates at a depth avoids duplicate combinations.

Q089. Generate valid parentheses for n pairs.

✎ SIMPLE ANSWER

Add an opening bracket while fewer than n are used, and a closing bracket only while fewer closes than opens are used. These constraints ensure every prefix remains valid.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
n=2	Add an opening bracket while fewer than n are used, and a closing bracket only while fewer closes than opens are used.	"(())", "()()"

💡 Prefix validity prunes invalid sequences before generating their full strings.

JAVA 17 · SOLUTION — Source Q089 · 12 lines

```
static List<String> parentheses(int n) {
  if(n<0) throw new IllegalArgumentException();
  List<String> out=new ArrayList<>(); parenthesesDFS(n,0,0,new StringBuilder(),out); return out;
}
static void parenthesesDFS(int n,int open,int close,StringBuilder path,List<String> out) {
  if(close==n) { out.add(path.toString()); return; }
  if(open<n) { path.append('('); parenthesesDFS(n,open+1,close,path,out);
path.setLength(path.length()-1); }
  if(close<open) {
    path.append(')'); parenthesesDFS(n,open,close+1,path,out);
    path.setLength(path.length()-1);
  }
}
```

⏱ TIME & SPACE

Time/output O(n*C_n), where C_n is Catalan number; auxiliary space O(n).

⚠ INTERVIEW PITFALL

n=0 returns one empty sequence, not zero solutions.

FOLLOW-UP QUESTION · ANSWERED

How would you count solutions without enumerating them?

Use the Catalan recurrence C0=1 and Cn=sum(Ci*C(n-1-i)) over splits of the first matching pair. Dynamic programming computes counts without generating every parenthesis string; BigInteger avoids overflow for large n.

Q090. Count N-Queens placements.

✎ SIMPLE ANSWER

Place one queen per row and track occupied columns and both diagonal families. Backtrack after each recursive choice.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
n=4	Place one queen per row and track occupied columns and both diagonal families.	2; n=0 -> 1 empty placement

💡 A valid choice cannot share a column, row-minus-column diagonal or row-plus-column diagonal.

JAVA 17 · SOLUTION — Source Q090 · 14 lines

```
static long queens(int n) {
  if(n<0 || n>12) throw new IllegalArgumentException();
  return queensDFS(n,0,new boolean[n],new boolean[2*n+1],new boolean[2*n+1]);
}
static long queensDFS(int n,int row,boolean[] col,boolean[] down,boolean[] up) {
  if(row==n) return 1; long count=0;
  for(int c=0;c<n;c++) {
    int d=row-c+n,u=row+c;
    if(col[c] || down[d] || up[u]) continue;
    col[c]=down[d]=up[u]=true; count+=queensDFS(n,row+1,col,down,up);
    col[c]=down[d]=up[u]=false;
  }
  return count;
}
```

⏱ TIME & SPACE

Exponential search, coarse O(n*n!) bound; auxiliary space O(n).

⚠ INTERVIEW PITFALL

The bound is an intentional exercise limit, not a general solver limit.

FOLLOW-UP QUESTION · ANSWERED

How could bit masks accelerate available-position calculation?

Represent used columns and both diagonal families as bit sets. At each row, available = mask & ~(columns | diagonals); pick set bits one at a time, updating shifted diagonal masks for the next row.

Q091. Count stair-climbing ways with steps of one or two.

✎ SIMPLE ANSWER

The last move comes from n-1 or n-2, so counts follow an adjacent-value recurrence. Use BigInteger to avoid primitive overflow and define one empty way for zero stairs.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
n=4	The last move comes from n-1 or n-2, so counts follow an adjacent-value recurrence.	ways(0)=ways(1)=1; ways(2)=2; ways(3)=3; ways(4)=5.

💡 ways(0)=ways(1)=1; ways(2)=2; ways(3)=3; ways(4)=5.

JAVA 17 · SOLUTION — Source Q091 · 8 lines

```
static BigInteger stairs(int n) {
  if(n<0) throw new IllegalArgumentException();
  BigInteger previous=BigInteger.ONE,current=BigInteger.ONE;
  for(int i=2;i<=n;i++) {
    BigInteger next=previous.add(current); previous=current; current=next;
  }
  return current;
}
```

⏱ TIME & SPACE

O(n) big-integer additions; storage and bit cost grow with result size.

⚠ INTERVIEW PITFALL

This convention counts the empty path at n=0.

FOLLOW-UP QUESTION · ANSWERED

How would allowed step sizes change the state transition?

Let ways[i] = sum(ways[i-step]) for each allowed positive step with i-step >= 0. Set ways[0]=1 and fill upward. Duplicated step sizes should not count as different moves unless specified.

Q092. Find the minimum coins for an amount.

✎ SIMPLE ANSWER

For each amount, consider taking one allowed coin after an already solved smaller amount. Use an unreachable sentinel and return -1 when no combination forms the target.

SKETCH THE PATTERN

EXAMPLE / INPUT
[1,2,5], amount=11

→

KEY LOGIC
For each amount, consider taking one allowed coin after an already solved smaller amount.

→

RESULT
Optimal substructure follows by removing the last selected coin.

💡 Optimal substructure follows by removing the last selected coin.

JAVA 17 · SOLUTION — Source Q092 · 8 lines

```
static int minCoins(int[] coins,int amount) {
  if(amount<0 || amount==Integer.MAX_VALUE) throw new IllegalArgumentException();
  for(int c:coins) if(c<=0) throw new IllegalArgumentException();
  int[] dp=new int[amount+1]; Arrays.fill(dp,amount+1); dp[0]=0;
  for(int a=1;a<=amount;a++) for(int c:coins)
    if(c<=a && dp[a-c]<=amount) dp[a]=Math.min(dp[a],dp[a-c]+1);
  return dp[amount]>amount ? -1 : dp[amount];
}
```

⏱ TIME & SPACE

Time O(amount*coinCount); space O(amount).

⚠ INTERVIEW PITFALL

Pseudo-polynomial in numeric amount; huge amounts can exhaust memory.

FOLLOW-UP QUESTION · ANSWERED

Why does greedy selection fail for coins [1,3,4] and amount 6?

Greedy takes 4+1+1, using three coins, but the optimal answer is 3+3, using two. General coin systems need DP or a proof of greedy optimality; always test coin-system assumptions.

Q093. Maximize nonadjacent house values.

✎ SIMPLE ANSWER

At each house, choose between skipping it and taking its value plus the best prefix that excludes the previous house. A rolling pair removes the need for a whole table.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[2,7,9,3,1]	At each house, choose between skipping it and taking its value plus the best prefix that excludes the previous house.	Taking 2,9,1 gives 12; adjacent choices cannot coexist.

💡 Taking 2,9,1 gives 12; adjacent choices cannot coexist.

JAVA 17 · SOLUTION — Source Q093 · 8 lines

```
static long rob(int[] values) {
  long previous=0,current=0;
  for(int value:values) {
    if(value<0) throw new IllegalArgumentException();
    long next=Math.max(current,previous+value); previous=current; current=next;
  }
  return current;
}
```

⏱ TIME & SPACE

Time O(n); auxiliary space O(1).

⚠ INTERVIEW PITFALL

Nonnegative int values; empty input gives zero.

FOLLOW-UP QUESTION · ANSWERED

How would a circular street split into two linear cases?

First and last houses cannot both be chosen. Compute the linear robbery optimum over [0,n-2] and [1,n-1], then take the larger. Handle n=1 separately.

Q094. Find the maximum sum of a nonempty contiguous subarray.

✎ SIMPLE ANSWER

Kadane's recurrence either extends the best subarray ending at the previous position or starts at the current value. Track the global best separately and use long sums.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[-2,1,-3,4,-1,2,1,-5,4]	Kadane's recurrence either extends the best subarray ending at the previous position or starts at the current value.	The winning segment is [4,-1,2,1], sum six.

💡 The winning segment is [4,-1,2,1], sum six.

JAVA 17 · SOLUTION — Source Q094 · 8 lines

```
static long maxSubarray(int[] a) {
  if(a.length==0) throw new IllegalArgumentException();
  long ending=a[0],best=a[0];
  for(int i=1;i<a.length;i++) {
    ending=Math.max((long)a[i],ending+a[i]); best=Math.max(best,ending);
  }
  return best;
}
```

⏱ TIME & SPACE

Time O(n); auxiliary space O(1).

⚠ INTERVIEW PITFALL

Nonempty contract avoids incorrectly returning zero for all-negative input.

FOLLOW-UP QUESTION · ANSWERED

How would you return start and end indices?

Track a candidate start for the current Kadane sum. When starting at a[i] beats extending, reset the candidate start to i; whenever a better maximum appears, store candidateStart and i as answer indices.

Q095. Find length of the longest strictly increasing subsequence.

✎ SIMPLE ANSWER

Maintain the smallest possible tail for every attained length. Replace the first tail at least as large as the current value; append when all tails are smaller.

SKETCH THE PATTERN

EXAMPLE / INPUT
[10,9,2,5,3,7,101,18]

→

KEY LOGIC
Maintain the smallest possible tail for every attained length.

→

RESULT
A smaller tail preserves more opportunities for future extension.

💡 A smaller tail preserves more opportunities for future extension.

JAVA 17 · SOLUTION — Source Q095 · 9 lines

```
static int lis(int[] a) {
  int[] tails=new int[a.length]; int size=0;
  for(int x:a) {
    int l=0,r=size;
    while(l<r) { int m=l+(r-l)/2; if(tails[m]<x) l=m+1; else r=m; }
    tails[l]=x; if(l==size) size++;
  }
  return size;
}
```

⏱ TIME & SPACE

Time O(n log(n+1)); space O(n).

⚠ INTERVIEW PITFALL

Equal values replace rather than extend because increasing is strict.

FOLLOW-UP QUESTION · ANSWERED

What extra predecessor metadata reconstructs a subsequence?

For each input index, store the predecessor index of the best subsequence ending there, and remember the tail index for each length. Backtrack from the longest tail, then reverse the collected indices/values.

Q096. Find longest common subsequence length.

✎ SIMPLE ANSWER

Use a table for prefix pairs. Equal final characters extend the smaller-prefix solution; otherwise discard one final character and take the better result.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
"abcde", "ace"	Use a table for prefix pairs.	A subsequence may skip positions; it need not be contiguous.

💡 A subsequence may skip positions; it need not be contiguous.

JAVA 17 · SOLUTION Source Q096 · 7 lines

```
static int lcs(String a,String b) {
  int[][] dp=new int[a.length()+1][b.length()+1];
  for(int i=1;i<=a.length();i++) for(int j=1;j<=b.length();j++)
    dp[i][j]=a.charAt(i-1)==b.charAt(j-1) ? dp[i-1][j-1]+1
        : Math.max(dp[i-1][j],dp[i][j-1]);
  return dp[a.length()][b.length()];
}
```

⏱ TIME & SPACE

Time O(n*m); space O((n+1)*(m+1)).

⚠ INTERVIEW PITFALL

Unicode code-point semantics require conversion before the recurrence.

FOLLOW-UP QUESTION · ANSWERED

How would rolling rows reduce space, and what does reconstruction need?

Only the previous DP row is needed to compute LCS length, reducing auxiliary memory to O(min(n,m)). To reconstruct a subsequence, keep choices/full DP or use Hirschberg divide-and-conquer reconstruction.

Q097. Compute edit distance.

✎ SIMPLE ANSWER

Solve all prefix pairs with insertion, deletion and replacement transitions. Equal final characters add no cost; other choices cost one.

SKETCH THE PATTERN

EXAMPLE / INPUT
"kitten", "sitting"

→

KEY LOGIC
Solve all prefix pairs with insertion, deletion and replacement transitions.

→

RESULT
Base row/column represent inserting/deleting an entire prefix.

💡 Base row/column represent inserting/deleting an entire prefix.

JAVA 17 · SOLUTION Source Q097 · 10 lines

```
static int editDistance(String a,String b) {
  int[][] dp=new int[a.length()+1][b.length()+1];
  for(int i=0;i<=a.length();i++) dp[i][0]=i;
  for(int j=0;j<=b.length();j++) dp[0][j]=j;
  for(int i=1;i<=a.length();i++) for(int j=1;j<=b.length();j++) {
    if(a.charAt(i-1)==b.charAt(j-1)) dp[i][j]=dp[i-1][j-1];
    else dp[i][j]=1+Math.min(dp[i-1][j-1],Math.min(dp[i-1][j],dp[i][j-1]));
  }
  return dp[a.length()][b.length()];
}
```

⏱ TIME & SPACE

Time O(n*m); space O((n+1)*(m+1)).

⚠ INTERVIEW PITFALL

Transposition is not one operation under this contract.

FOLLOW-UP QUESTION · ANSWERED

How would weighted operations or Damerau distance change the recurrence?

Replace the unit insertion/deletion/substitution costs with the selected weights in the recurrence. Damerau-Levenshtein also allows transposition of adjacent characters, requiring additional neighboring states and a precise variant definition.

Q098. Check whether a string can be segmented into dictionary words.

✎ SIMPLE ANSWER

Mark prefix positions reachable by valid dictionary pieces. A reachable earlier prefix plus a matching next substring makes a later prefix reachable.

SKETCH THE PATTERN

EXAMPLE / INPUT
"leetcode", {"leet","code"}

→

KEY LOGIC
Mark prefix positions reachable by valid dictionary pieces.

→

RESULT
true

💡 Only reachable prefix boundaries can begin the next word.

JAVA 17 · SOLUTION Source Q098 · 6 lines

```
static boolean wordBreak(String text,Set<String> dictionary) {
  boolean[] dp=new boolean[text.length()+1]; dp[0]=true;
  for(int end=1;end<=text.length();end++) for(int start=0;start<end;start++)
    if(dp[start] && dictionary.contains(text.substring(start,end))) { dp[end]=true; break; }
  return dp[text.length()];
}
```

⏱ TIME & SPACE

O(n squared) transitions; Java substring/copy/hash cost makes worst O(n cubed) character work. O(n) DP plus transient substrings.

⚠ INTERVIEW PITFALL

This simple Java version must not claim constant-time substring construction.

FOLLOW-UP QUESTION · ANSWERED

How would a trie or maximum dictionary-word length limit work?

A trie lets each reachable position walk only valid dictionary prefixes, avoiding some substring allocations. Alternatively bound transition length by the longest dictionary word. Worst-case cost still depends on input and dictionary shape.

Q099. Solve 0/1 knapsack with a one-dimensional table.

✎ SIMPLE ANSWER

For each item, update capacities backward so each item contributes at most once. If iterating forward, this recurrence could reuse the same item.

SKETCH THE PATTERN

EXAMPLE / INPUT
weights [2,3,4], values [4,5,7], capacity 5

→

KEY LOGIC
For each item, update capacities backward so each item contributes at most once.

→

RESULT
Descending capacity uses the previous item set for the referenced sm

💡 Descending capacity uses the previous item set for the referenced smaller capacity.

JAVA 17 · SOLUTION Source Q099 · 10 lines

```
static long knapsack(int[] weights,int[] values,int capacity) {
  if(capacity<0 || weights.length!=values.length) throw new IllegalArgumentException();
  long[] dp=new long[capacity+1];
  for(int i=0;i<weights.length;i++) {
    if(weights[i]<0 || values[i]<0) throw new IllegalArgumentException();
    for(int c=capacity;c>=weights[i];c--)
      dp[c]=Math.max(dp[c],dp[c-weights[i]]+values[i]);
  }
  return dp[capacity];
}
```

⏱ TIME & SPACE

Time O(itemCount*capacity); space O(capacity).

⚠ INTERVIEW PITFALL

Pseudo-polynomial; zero-weight items are included at most once; impractically large capacity is outside the resource scope.

FOLLOW-UP QUESTION · ANSWERED

How would unbounded knapsack change iteration order?

For unbounded knapsack, iterate capacity upward for each item so the current item's new result can be reused in the same pass. In 0/1 knapsack, iterate downward to prevent selecting the same item twice.

Q100. Count paths in an obstacle-free grid.

✎ SIMPLE ANSWER

Use one row of BigInteger counts. Each cell is reached from above or left; updating left-to-right combines those contributions.

SKETCH THE PATTERN

EXAMPLE / INPUT
3 rows, 7 columns

→

KEY LOGIC
Use one row of BigInteger counts.

→

RESULT
The top row and first column each have one path.

💡 The top row and first column each have one path.

JAVA 17 · SOLUTION Source Q100 · 6 lines

```
static BigInteger gridPaths(int rows,int cols) {
  if(rows<1 || cols<1) throw new IllegalArgumentException();
  BigInteger[] dp=new BigInteger[cols]; Arrays.fill(dp,BigInteger.ONE);
  for(int r=1;r<rows;r++) for(int c=1;c<cols;c++) dp[c]=dp[c].add(dp[c-1]);
  return dp[cols-1];
}
```

⏱ TIME & SPACE

O(rows*cols) big-integer additions; space O(cols) numbers with growing bit length.

⚠ INTERVIEW PITFALL

No obstacles and no diagonal moves; exact counts can exceed long.

FOLLOW-UP QUESTION · ANSWERED

How do obstacles change initialization and cell updates?

Set blocked cells' path count to zero. Initialize the start cell to one only if open, and for each other open cell add ways from top and left. Obstacles prevent any path from flowing through that position.

Q101. Count words with Java streams and preserve first-seen order.

✎ SIMPLE ANSWER

Use groupingBy with LinkedHashMap and counting. The input is already tokenized; this version is case-sensitive and treats each non-null string as one key.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
["red","blue","red"]	Use groupingBy with LinkedHashMap and counting.	red=2, blue=1

💡 The map factory selects encounter-key order instead of unspecified HashMap iteration.

JAVA 17 · SOLUTION — Source Q101 · 4 lines

```
static Map<String,Long> wordFrequency(List<String> words) {
  return words.stream().collect(Collectors.groupingBy(
      Function.identity(),LinkedHashMap::new,Collectors.counting()));
}
```

⏱ TIME & SPACE

Expected time O(n plus key hashing); space O(u).

⚠ INTERVIEW PITFALL

Tokenization, punctuation and case normalization are outside this method.

FOLLOW-UP QUESTION · ANSWERED

How would you deliberately normalize using Locale.ROOT?

Define normalization explicitly, for example word.toLowerCase(Locale.ROOT), before grouping or counting. Locale.ROOT gives locale-independent Unicode casing but may not replace full normalization or case folding requirements.

Q102. List repeated words in sorted order.

✎ SIMPLE ANSWER

Group frequencies into a TreeMap, keep counts above one and collect keys. This returns distinct repeated words in natural String ordering, not every repeated occurrence.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
["b","a","b","a","c"]	Group frequencies into a TreeMap, keep counts above one and collect keys.	["a","b"]

💡 Grouping selects duplicates; sorted map order determines result order.

JAVA 17 · SOLUTION — Source Q102 · 5 lines

```
static List<String> duplicateWords(List<String> words) {
  return words.stream().collect(Collectors.groupingBy(
      Function.identity(),TreeMap::new,Collectors.counting()))
      .entrySet().stream().filter(e->e.getValue()>1).map(Map.Entry::getKey).toList();
}
```

⏱ TIME & SPACE

O(n log(u+1)) comparisons plus string comparison cost; space O(u), excluding output.

⚠ INTERVIEW PITFALL

Java 17 toList returns an unmodifiable list; keys are case-sensitive.

FOLLOW-UP QUESTION · ANSWERED

When would a HashMap plus final sorting be cheaper?

A HashMap offers lower insertion overhead than maintaining TreeMap sorted order during accumulation. If there are many tokens but few unique words, count in O(n) expected time and sort only the u final keys in O(u log u).

Q103. Group employees by department.

✎ SIMPLE ANSWER

Use a grouping collector with TreeMap for sorted department keys and toList for employees in encounter order. Grouping does not automatically copy or deeply freeze every value object.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Employees from A,B,A	Use a grouping collector with TreeMap for sorted department keys and toList for employees in encounter order.	A has two entries, B one

💡 The classifier creates a key; the downstream collector accumulates each group.

JAVA 17 · SOLUTION — Source Q103 · 4 lines

```
static Map<String,List<Employee>> groupDepartment(List<Employee> employees) {
  return employees.stream().collect(Collectors.groupingBy(
      Employee::department,TreeMap::new,Collectors.toList()));
}
```

⏱ TIME & SPACE

O(n log(d+1)) department-key comparisons; space O(n+d).

⚠ INTERVIEW PITFALL

Employee is an immutable shared record; returned map/lists are not promised immutable.

FOLLOW-UP QUESTION · ANSWERED

How would you make the returned group structure immutable?

Copy each group's list with List.copyOf, then copy the map with Map.copyOf or an unmodifiable ordered map if iteration order matters. This protects collection structure; mutable Employee elements would need deeper copying.

Q104. Find the highest-paid employee per department.

✎ SIMPLE ANSWER

Collect by department and supply a merge function that retains the greater salary. For equal salaries, retain the first employee encountered.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
A: Ana=100, Bo=200	Collect by department and supply a merge function that retains the greater salary.	A maps to Bo

💡 A duplicate department invokes a deliberate resolution rule instead of throwing.

JAVA 17 · SOLUTION — Source Q104 · 4 lines

```
static Map<String,Employee> highestPaid(List<Employee> employees) {
  return employees.stream().collect(Collectors.toMap(Employee::department,
      Function.identity(),(a,b)->a.salary()>=b.salary() ? a : b,TreeMap::new));
}
```

⏱ TIME & SPACE

O(n log(d+1)) key comparisons; space O(d).

⚠ INTERVIEW PITFALL

Tie policy matters; monetary rounding is a separate BigDecimal task.

FOLLOW-UP QUESTION · ANSWERED

How would you return all tied highest-paid employees?

First find each department's maximum salary, then filter employees whose salary matches their department maximum. Return a list per department rather than using maxBy, which selects only one tied employee.

Q105. Flatten nested lists and remove duplicates in encounter order.

✎ SIMPLE ANSWER

flatMap emits inner values, and distinct keeps the first occurrence of each value in an ordered stream. Collect with toList for an unmodifiable Java 17 result.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[[1,2],[2,3],[]]	flatMap emits inner values, and distinct keeps the first occurrence of each value in an ordered stream.	[1,2,3]

💡 Flattening removes one container level; distinct then applies value equality.

JAVA 17 · SOLUTION — Source Q105 · 3 lines

```
static List<Integer> flattenDistinct(List<List<Integer>> input) {
  return input.stream().flatMap(Collection::stream).distinct().toList();
}
```

⏱ TIME & SPACE

Expected O(total elements); space O(u), excluding output.

⚠ INTERVIEW PITFALL

Nested lists and elements follow the non-null guide contract.

FOLLOW-UP QUESTION · ANSWERED

How would changing encounter order affect which duplicate object is retained?

Distinct keeps the first equivalent item in the stream's encounter order. Reordering inputs or calling unordered() can change which object instance is retained when equal objects carry different non-equality metadata.

Q106. Find the second-highest distinct salary with streams.

✎ SIMPLE ANSWER

Extract salary values, remove duplicates, sort descending, skip the highest and return the next value. Absence handles an empty input or only one distinct salary.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[100,200,200,150]	Extract salary values, remove duplicates, sort descending, skip the highest and return the next value.	150

💡 Distinct ranking differs from ranking employees with duplicate salaries.

JAVA 17 · SOLUTION — Source Q106 · 4 lines

```
static OptionalInt secondSalary(List<Employee> employees) {
  return employees.stream().map(Employee::salary).distinct()
      .sorted(Comparator.reverseOrder()).skip(1).mapToInt(Integer::intValue).findFirst();
}
```

⏱ TIME & SPACE

Expected O(n + u log u) time; space O(u).

⚠ INTERVIEW PITFALL

The solution allocates distinct/sort state and boxes salaries.

FOLLOW-UP QUESTION · ANSWERED

How could two running distinct maxima avoid sorting?

Scan once, maintaining Integer first and second distinct maxima. Update second when a new highest arrives or when an intermediate distinct value improves second. O(n) time and O(1) extra state.

Q107. Sort employees by department, descending salary, then name.

✎ SIMPLE ANSWER

Compose comparators in the order the specification requires. Apply reversed only to the salary comparator, not the whole chain.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
A employees come first; A: salary 200 before 100	Compose comparators in the order the specification requires.	A employees come first; A: salary 200 before 100

💡 A tie reaches the next comparator; salary reversal leaves department order ascending.

JAVA 17 · SOLUTION Source Q107 · 6 lines

```
static List<Employee> sortEmployees(List<Employee> employees) {
  Comparator<Employee> order=Comparator.comparing(Employee::department)
      .thenComparing(Comparator.comparingInt(Employee::salary).reversed())
      .thenComparing(Employee::name);
  return employees.stream().sorted(order).toList();
}
```

⏱ TIME & SPACE

Time O(n log n) comparisons plus String comparison costs; space O(n).

⚠ INTERVIEW PITFALL

Names/departments are non-null; stability resolves fully equal comparator keys.

FOLLOW-UP QUESTION · ANSWERED

How would explicit null handling change the order?

Specify Comparator.nullsFirst/nullsLast for nullable department or name fields, and decide how null salary records are handled. Without an explicit policy, the normal comparing chain may throw NullPointerException.

Q108. Partition integers into even and odd groups.

✎ SIMPLE ANSWER

partitioningBy produces true and false groups based on a predicate. Use remainder equals zero so negative even integers are also recognized.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[-2,-1,0,3]	partitioningBy produces true and false groups based on a predicate.	true:[-2,0], false:[-1,3]

💡 Partition has two logical buckets rather than one bucket per distinct value.

JAVA 17 · SOLUTION — Source Q108 · 3 lines

```
static Map<Boolean,List<Integer>> partitionEven(List<Integer> values) {
  return values.stream().collect(Collectors.partitioningBy(x->x%2==0));
}
```

⏱ TIME & SPACE

Time O(n); space O(n).

⚠ INTERVIEW PITFALL

Both boolean keys exist even for empty input; null values are rejected by the guide contract.

FOLLOW-UP QUESTION · ANSWERED

When would groupingBy be the better abstraction?

Use groupingBy when you need a Map<Boolean,List<Integer>>, downstream collectors, or future group-specific calculations. partitioningBy is clearer for exactly two boolean groups, and always supplies both keys.

Q109. Aggregate salary totals by department without int-sum overflow.

✎ SIMPLE ANSWER

Use toMap with a long-valued mapper and sum merge function. Explicit duplicate handling is required because several employees share the same department.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
A:100+200	Use toMap with a long-valued mapper and sum merge function.	300; B:100 -> 100

💡 Widen each salary before aggregation rather than widening an already-overflowed int sum.

JAVA 17 · SOLUTION Source Q109 · 4 lines

```
static Map<String,Long> salaryTotals(List<Employee> employees) {
  return employees.stream().collect(Collectors.toMap(Employee::department,
      e->(long)e.salary(),Long::sum,TreeMap::new));
}
```

⏱ TIME & SPACE

O(n log(d+1)) key comparisons; space O(d).

⚠ INTERVIEW PITFALL

Use numeric units consistently; this is not a currency-conversion routine.

FOLLOW-UP QUESTION · ANSWERED

How would groupingBy with summingLong express the same result?

employees.stream().collect(groupingBy(Employee::department, summingLong(Employee::salary))) groups employees by department and sums into long values. This prevents 32-bit intermediate salary-sum overflow for normal input sizes.

Q110. Sum int values with a safe parallel reduction.

✎ SIMPLE ANSWER

Map each value to long and use the built-in sum reduction. It avoids a shared mutable accumulator and has an associative identity/combination for sums in the stated finite int-array domain.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[1,2,3]	Map each value to long and use the built-in sum reduction.	Workers compute partial sums and combine them rather than racing on

💡 Workers compute partial sums and combine them rather than racing on one count field.

JAVA 17 · SOLUTION Source Q110 · 3 lines

```
static long parallelSum(int[] values) {
  return Arrays.stream(values).parallel().mapToLong(x->x).sum();
}
```

⏱ TIME & SPACE

Work O(n); parallel runtime and internal storage depend on execution/splitting. No speedup claimed.

⚠ INTERVIEW PITFALL

A Java int array's full sum fits long; this guarantee does not cover arbitrary long inputs.

FOLLOW-UP QUESTION · ANSWERED

Why can a floating-point parallel sum differ from sequential rounding?

Floating-point addition is not associative: (a+b)+c may round differently from a+(b+c). Parallel reductions group terms differently, so finite sums may differ slightly from sequential order even when the reduction is otherwise valid.

Q111. Parse and validate a strict calendar date.

✎ SIMPLE ANSWER

Use an explicit uuuu-based formatter with STRICT resolution so impossible dates fail. Year-of-era yyyy has different resolution semantics and should not be substituted accidentally.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
"2024-02-29" valid; "2023-02-29" invalid	Use an explicit uuuu-based formatter with STRICT resolution so impossible dates fail.	"2024-02-29" valid; "2023-02-29" invalid

💡 Strict resolution verifies the calendar rather than adjusting an invalid date.

JAVA 17 · SOLUTION — Source Q111 · 5 lines

```
static LocalDate strictDate(String text) {
  DateTimeFormatter formatter=DateTimeFormatter.ofPattern("uuuu-MM-dd",Locale.ROOT)
      .withResolverStyle(ResolverStyle.STRICT);
  return LocalDate.parse(text,formatter);
}
```

⏱ TIME & SPACE

Time O(text length) parsing; bounded formatter/state storage for this format.

⚠ INTERVIEW PITFALL

LocalDate is a date without a time zone or instant.

FOLLOW-UP QUESTION · ANSWERED

How would a zoned event need Instant or ZonedDateTime instead?

LocalDate represents a calendar day without time zone. Use ZonedDateTime for a local event in a named ZoneId and Instant for a unique timeline point; account for DST gaps and overlapping local times.

Q112. Add decimal amounts and round once at the result boundary.

✎ SIMPLE ANSWER

Parse decimal strings into BigDecimal, sum exactly, then round the aggregate to two places using a specified policy. This contract deliberately rounds the total rather than every input item.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
["0.10","0.20"]	Parse decimal strings into BigDecimal, sum exactly, then round the aggregate to two places using a specified policy.	0.30

💡 Decimal text avoids introducing binary floating-point approximation before aggregation.

JAVA 17 · SOLUTION — Source Q112 · 5 lines

```
static BigDecimal totalMoney(List<String> amounts) {
  BigDecimal total=BigDecimal.ZERO;
  for(String amount:amounts) total=total.add(new BigDecimal(amount));
  return total.setScale(2,RoundingMode.HALF_EVEN);
}
```

⏱ TIME & SPACE

Time depends on total digits and BigDecimal arithmetic; storage grows with numeric representation.

⚠ INTERVIEW PITFALL

Rounding per item can produce a different total; currency/scale policy must be specified separately.

FOLLOW-UP QUESTION · ANSWERED

Why do equals and compareTo treat scale differently?

BigDecimal.equals checks value and scale, so 2.0 != 2.00 by equals; compareTo compares numeric value and returns zero for these. Use compareTo for amount ordering and deliberate scale policy for keys or serialization.

Q113. Create an immutable snapshot of mutable input lists.

✎ SIMPLE ANSWER

Make defensive copies of both the outer list and each inner list. Integer elements are immutable, so copying structure suffices for this contract.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Changing the original inner list must not alter the snapshot.	Make defensive copies of both the outer list and each inner list.	Changing the original inner list must not alter the snapshot.

💡 Each inner copy detaches structural updates; outer toList forbids outer modification.

JAVA 17 · SOLUTION — Source Q113 · 3 lines

```
static List<List<Integer>> immutableSnapshot(List<List<Integer>> input) {
  return input.stream().map(List::copyOf).toList();
}
```

⏱ TIME & SPACE

Time/space O(number of lists + total elements).

⚠ INTERVIEW PITFALL

This does not deep-copy mutable custom element objects; null elements are rejected by List.copyOf.

FOLLOW-UP QUESTION · ANSWERED

What would a snapshot of mutable Address objects need?

Create new Address instances or immutable Address values for every element before collecting into List.copyOf. Copying only the outer list does not freeze mutable objects referenced inside it.

Q114. Design a stable composite map key.

✎ SIMPLE ANSWER

An immutable record generates value-based equality and hash code from its components. Use only components with stable equality while the key is stored.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Key("IN",7) equals another Key("IN",7)	An immutable record generates value-based equality and hash code from its components.	Key("IN",7) equals another Key("IN",7)

💡 Equal keys can retrieve the same entry despite being different object instances.

JAVA 17 · SOLUTION — Source Q114 · 8 lines

```
record CustomerKey(String country,long id) {
  CustomerKey { Objects.requireNonNull(country); }
}
static String keyLookup() {
  Map<CustomerKey,String> values=new HashMap<>();
  values.put(new CustomerKey("IN",7),"customer");
  return values.get(new CustomerKey("IN",7));
}
```

⏱ TIME & SPACE

Expected O(1) map access plus String hashing/equality; storage O(entries).

⚠ INTERVIEW PITFALL

A record with mutable components would not automatically be a safe stable key.

FOLLOW-UP QUESTION · ANSWERED

How would you implement equals/hashCode without records?

Define an immutable key class, mark fields final, compare every identity field in equals, and compute hashCode using those same fields via Objects.hash or a manual combination. Never mutate a key after insertion.

Q115. Sort integer extremes without comparator overflow.

✎ SIMPLE ANSWER

Use Integer.compare through comparingInt instead of subtraction. Subtracting keys can overflow and violate comparator ordering when inputs include opposite integer extremes.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
[MAX_VALUE,MIN_VALUE,0]	Use Integer.compare through comparingInt instead of subtraction.	[MIN_VALUE,0,MAX_VALUE]

💡 MIN_VALUE must remain smaller than MAX_VALUE under every comparison.

JAVA 17 · SOLUTION Source Q115 · 3 lines

```
static List<Integer> safeSort(List<Integer> input) {
  List<Integer> out=new ArrayList<>(input); out.sort(Integer::compare); return out;
}
```

⏱ TIME & SPACE

Time O(n log n); space O(n), including copy/sorting storage.

⚠ INTERVIEW PITFALL

Input remains unchanged; null ordering is outside this method's contract.

FOLLOW-UP QUESTION · ANSWERED

Why can a broken comparator violate sort algorithm assumptions?

A comparator must be antisymmetric, transitive and consistent with its equality policy. Subtracting extreme ints can overflow and reverse ordering, causing broken sort behavior or exceptions. Use Integer.compare(a,b) instead.

Q116. Count concurrent increments exactly.

✎ SIMPLE ANSWER

Use an AtomicInteger for each indivisible increment, retain every submitted Future and observe completion before reading the final total. Always shut down the owned executor.

SKETCH THE PATTERN

EXAMPLE / INPUT
4 tasks x 1000 increments

→

KEY LOGIC
Use an AtomicInteger for each indivisible increment, retain every submitted Future and observe completion before reading the final...

→

RESULT
4000

💡 Atomic update prevents lost increments; get supplies completed-task observation.

JAVA 17 · SOLUTION — Source Q116 · 14 lines

```
static int concurrentCount(int tasks,int repetitions) throws Exception {
  if(tasks<1 || repetitions<0 || (long)tasks*repetitions>Integer.MAX_VALUE)
    throw new IllegalArgumentException();
  ExecutorService pool=Executors.newFixedThreadPool(Math.min(tasks,4));
  AtomicInteger count=new AtomicInteger();
  try {
    List<Future<?>> futures=new ArrayList<>();
    for(int i=0;i<tasks;i++) futures.add(pool.submit(()-> {
      for(int j=0;j<repetitions;j++) count.incrementAndGet();
    }));
    for(Future<?> future:futures) future.get(5,TimeUnit.SECONDS);
    return count.get();
  } finally { stopPool(pool); }
}
```

⏱ TIME & SPACE

Work O(tasks*repetitions); stored Futures O(tasks); contention cost is workload-dependent.

⚠ INTERVIEW PITFALL

The exercise is bounded, not a throughput benchmark; stopPool is a shared lifecycle helper.

FOLLOW-UP QUESTION · ANSWERED

When is LongAdder appropriate instead of an exact decision counter?

LongAdder suits high-contention statistics when sum need not be an atomic live snapshot. Exact quotas, ticket numbers and admission decisions require an AtomicInteger/AtomicLong or a lock guarding the full invariant.

Q117. Transfer integers through a bounded producer-consumer queue.

✎ SIMPLE ANSWER

Use two owned workers and a bounded BlockingQueue. A reserved negative stop marker follows all nonnegative values.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Values 1 through 100	Use two owned workers and a bounded BlockingQueue.	sum 5050

💡 FIFO places the terminal marker after every value from this single producer.

JAVA 17 · SOLUTION — Source Q117 · 15 lines

```
static long queueSum(int n) throws Exception {
  if(n<0 || n==Integer.MAX_VALUE) throw new IllegalArgumentException();
  BlockingQueue<Integer> queue=new ArrayBlockingQueue<>(8);
  ExecutorService pool=Executors.newFixedThreadPool(2);
  try {
    Future<Long> consumer=pool.submit(()-> {
      long total=0;
      while(true) { int x=queue.take(); if(x==-1) return total; total+=x; }
    });
    Future<?> producer=pool.submit(()-> {
      for(int i=1;i<=n;i++) queue.put(i); queue.put(-1); return null;
    });
    producer.get(5,TimeUnit.SECONDS); return consumer.get(5,TimeUnit.SECONDS);
  } finally { stopPool(pool); }
}
```

⏱ TIME & SPACE

Work O(n); queue space O(1) fixed capacity; two worker resources.

⚠ INTERVIEW PITFALL

MAX_VALUE is rejected to avoid counter wrap; practical n must finish within the exercise deadline.

FOLLOW-UP QUESTION · ANSWERED

How does termination change with several consumers?

Send one termination marker per consumer only after all producer work is complete, or use coordinated interruption/queue close semantics. Otherwise one consumer may stop while others remain blocked indefinitely.

Q118. Run independent calculations and combine completion results.

✎ SIMPLE ANSWER

Supply an explicitly owned executor and combine results through thenCombine. Avoid shared mutable side channels.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
20 and 22	Supply an explicitly owned executor and combine results through thenCombine.	Combination depends on both normal completions and uses their return

💡 Combination depends on both normal completions and uses their returned values.

JAVA 17 · SOLUTION Source Q118 · 8 lines

```
static long combineAsync(int a,int b) throws Exception {
  ExecutorService pool=Executors.newFixedThreadPool(2);
  try {
    CompletableFuture<Integer> left=CompletableFuture.supplyAsync(()->a,pool);
    CompletableFuture<Integer> right=CompletableFuture.supplyAsync(()->b,pool);
    return left.thenCombine(right,(x,y)->(long)x+y).get(5,TimeUnit.SECONDS);
  } finally { stopPool(pool); }
}
```

⏱ TIME & SPACE

O(1) synthetic work; scheduling/executor overhead dominates; bounded task resources.

⚠ INTERVIEW PITFALL

Timed get is not a universal cancellation guarantee; the tasks here do not block externally.

FOLLOW-UP QUESTION · ANSWERED

How would you recover one failed dependency while preserving semantics?

Attach exception handling to the dependency stage that is optional, replacing its result with a documented fallback. Use thenCombine for independent results, and never silently turn a required payment/authorization failure into success.

Q119. Verify executor tasks run in FIFO order with one worker.

✎ SIMPLE ANSWER

Use a single-worker executor and submit tasks that append to an owned list. Retrieve every completion before copying the output.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
Submit 0,1,2	Use a single-worker executor and submit tasks that append to an owned list.	output [0,1,2]

💡 One worker consumes the factory's FIFO work queue; completion handoff precedes observer copying.

JAVA 17 · SOLUTION — Source Q119 · 11 lines

```
static List<Integer> singleWorkerOrder(int count) throws Exception {
  if(count<0) throw new IllegalArgumentException();
  ExecutorService pool=Executors.newSingleThreadExecutor();
  List<Integer> output=new ArrayList<>();
  try {
    List<Future<?>> futures=new ArrayList<>();
    for(int i=0;i<count;i++) { int value=i; futures.add(pool.submit(()->output.add(value))); }
    for(Future<?> future:futures) future.get(5,TimeUnit.SECONDS);
    return List.copyOf(output);
  } finally { stopPool(pool); }
}
```

⏱ TIME & SPACE

Work O(count); space O(count) for queued tasks, handles and output.

⚠ INTERVIEW PITFALL

The factory queue is unbounded; this finite demonstration is not a production admission design.

FOLLOW-UP QUESTION · ANSWERED

Would a multiworker executor preserve completion order?

No. Multiple workers can start and complete tasks in a different order because task durations and scheduling vary. A single-worker FIFO queue normally orders task start; use ordered result collection when output order is required.

Q120. Bound concurrent work with permits and verify the peak.

✎ SIMPLE ANSWER

Acquire a permit before counting active work and release it in finally. Track the highest observed active value with an atomic maximum.

SKETCH THE PATTERN

EXAMPLE / INPUT	KEY LOGIC	RESULT
20 tasks, permits=3	Acquire a permit before counting active work and release it in finally.	observed peak between 1 and 3

💡 Every active increment is inside an acquired-permit lifecycle.

JAVA 17 · SOLUTION — Source Q120 · 17 lines

```
static int boundedPeak(int tasks,int limit) throws Exception {
  if(tasks<1 || limit<1) throw new IllegalArgumentException();
  Semaphore permits=new Semaphore(limit); AtomicInteger active=new AtomicInteger(),peak=new
AtomicInteger();
  ExecutorService pool=Executors.newFixedThreadPool(Math.min(tasks,8));
  try {
    List<Future<?>> futures=new ArrayList<>();
    for(int i=0;i<tasks;i++) futures.add(pool.submit(()-> {
      permits.acquire();
      try { int now=active.incrementAndGet(); peak.accumulateAndGet(now,Math::max); }
      finally { active.decrementAndGet(); permits.release(); }
      return null;
    }));
    for(Future<?> f:futures) f.get(5,TimeUnit.SECONDS);
    if(active.get()!=0 || permits.availablePermits()!=limit) throw new AssertionError();
    return peak.get();
  } finally { stopPool(pool); }
}
```

⏱ TIME & SPACE

Work O(tasks); stored handles O(tasks), at most eight worker threads.

⚠ INTERVIEW PITFALL

Peak may be below the limit because work is tiny and scheduling varies.

FOLLOW-UP QUESTION · ANSWERED

How would rejection or cancellation before start affect permit ownership?

Acquire permits only inside tasks that actually begin, or track reservation state and release exactly once on rejection, pre-start cancellation, failure, or completion. A permit leaked during cancellation permanently lowers usable capacity.

QUICK REFERENCE

Pick the right coding pattern

Spot the input shape, name the invariant, then choose a tool.

Two pointers
Sorted data or two-ended comparison

Move one end only when sorted-order reasoning proves that end cannot help.

Sliding window
Contiguous segment, bounded condition

Expand; shrink when required, maintaining valid state.

Hash map / frequency
Repeated values, complements, prefixes

Remember earlier observations without rescanning.

Binary search
Monotonic yes/no predicate

Preserve a boundary invariant; test empty and off-by-one cases.

Monotonic stack
Next larger/smaller; unresolved indices

Keep candidate indices in monotone order and pop when resolved.

Heap
Repeated best k / top priority

Retain a bounded best-candidate set and define tie handling.

BFS / DFS
Graph reachability, tree and grid

Track visited identity; BFS gives minimum hops on unweighted graphs.

Dynamic programming
Overlapping choices and recurrence

Write a state and transition before implementation.

Backtracking
All valid combinations under constraints

Choose; explore; restore branch state, with small input bounds.

Java Streams
Collection transforms, aggregation

State encounter order, duplicates, nulls, mutability and numeric type.

Thread synchronization
Concurrent operations with strict state

Name the atomic invariant, handoff and cleanup path.

Overflow & precision
BigInteger, BigDecimal, extreme ints

Prove numeric bounds or use exact types and safe comparators.

REVISION CHECKLIST

Interview explanation in 7 steps

1. **Define input and output.** Say what happens for empty, null (if allowed), negative, duplicate or overflow input.
2. **Start with a simple approach.** Discuss where it wastes time or memory.
3. **Name the invariant.** Explain what remains true after every loop or recursive call.
4. **Write a small Java implementation.** Choose names that reveal the algorithm.
5. **Trace one normal input.** Draw a few states or pointer moves.
6. **State time and auxiliary space.** Include copying, recursion stack and output conventions.
7. **Answer the follow-up.** Modify only what the new contract requires.

> Avoid memorizing code blindly. A precise explanation of why a pointer moves or a state updates is what makes a solution reusable.

30-DAY PLAN

One month to interview readiness

Days 1–3	Numbers, digits, strings	Q001–Q015
Days 4–6	Windows, arrays, hashing	Q016–Q030
Days 7–9	Pointers and binary search	Q031–Q040
Days 10–12	Sorting, matrices, stacks	Q041–Q055
Days 13–15	Heaps, linked lists	Q056–Q070
Days 16–18	Trees, BST, serialization	Q071–Q080
Days 19–21	Graphs and backtracking	Q081–Q090
Days 22–24	Dynamic programming	Q091–Q100
Days 25–27	Streams and Java correctness	Q101–Q115
Days 28–29	Concurrent code and follow-ups	Q116–Q120
Day 30	Three timed mock interview rounds	Explain patterns; rerun solutions

Every day's mini routine

45 minutes: attempt new problems · 25 minutes: trace edge cases · 25 minutes: explain follow-ups aloud · 15 minutes: fix errors from yesterday.

> For problems Q086–Q100, allow more time to define recurrence, pruning and state. For Q116–Q120, also verify cancellation, executor shutdown and resource ownership.

RUN LOCALLY

Java source & practice pack

The separate ZIP contains one assembled `InterviewSolutions.java` file, all 120 extracted solutions, a command-line runner and individual source snippets.

```
javac --release 17 InterviewSolutions.java
java InterviewSolutions
# PASS 124 example assertions
```

The reference solutions are Java 17. They contain record declarations and Stream.toList() calls that require more than a Java 8 compiler. The 124 locally rerun sample assertions are not the original complete differential test suite.

Know what tests prove

A passing example confirms behavior for tested inputs. It does not prove correctness for every graph size, schedule, Unicode boundary, numeric range or invalid input. Use an invariant plus edge-case tests.

> Commercial publishing: ensure you have permission to adapt and redistribute the supplied question bank and source code.